DC Housing Finance Agency (DCHFA) Bond Portfolio: A Full Data Science Project

Dataset. This project works with the DCHFA Portfolio open data export, a public record of 216 bond financed multifamily affordable housing properties located across Washington, DC. Each row in the source file represents one financed property and carries information about its location, its building characteristics, the terms of the bond that financed it, and whether the property participates in any deep affordability subsidy program. The Housing Finance Agency itself is an independent, quasi governmental instrumentality of the District of Columbia. Its role is to issue tax exempt and taxable bonds and use the proceeds to finance the construction, acquisition, and rehabilitation of affordable multifamily rental housing throughout the city. Every row in this dataset is therefore not just a data point but a real building somewhere in Washington, DC, financed through a real bond transaction, housing real tenants, many of whom qualify for that housing specifically because of income restrictions tied to the financing.

Why this dataset is worth a full project. Municipal open data exports are almost never analysis ready the moment you download them, and this one is a good example of exactly why. It has inconsistent spellings scattered across nearly every categorical column, abbreviations mixed together with full text inside the very same field, missing values scattered across more than half of its thirty three raw columns, and at least one batch of eight records carrying a date that is simply impossible on its face. That combination makes this dataset a genuinely useful vehicle for demonstrating the full data science workflow end to end, rather than skipping straight to the interesting modeling step the way a lot of tutorial datasets encourage. Specifically, this project walks through auditing data quality before trusting any number that comes out of the file, building a documented and fully repeatable cleaning pipeline rather than making one time manual edits that nobody could reproduce later, exploring the cleaned data both visually and statistically, testing formal hypotheses about relationships that appear in the data rather than simply eyeballing a chart and calling it a finding, building predictive models while being honest about their real limitations, and finally translating all of that analytical work into concrete recommendations that a housing finance analyst or asset manager inside DCHFA could genuinely act on.

Goal. Turn this messy municipal export into a clean, well documented, analysis ready dataset, and then use that dataset to answer five questions that a housing finance analyst or asset manager would genuinely want answered, rather than five questions invented purely to showcase a particular Python library:

1. What does the portfolio actually look like, in terms of the number and size of properties it covers, where those properties are located across the city, what kinds of buildings dominate the portfolio, and how old that building stock is?
2. Where is DCHFA's capital concentrated geographically and financially, and how has the pace and scale of its lending activity evolved across the roughly three decades this dataset spans?
3. Which specific loans in the portfolio carry near term refinancing or maturity risk that asset management should be actively watching right now, rather than discovering only when a bond actually comes due?
4. How much of the portfolio serves subsidized or deeply affordable housing beyond the underlying bond financing terms alone, and which specific programs are doing the bulk of that work?
5. Can property characteristics that are easy to observe, such as unit count, building type, ward, and construction era, meaningfully predict loan size and help segment the portfolio by risk, and just as importantly, how much confidence should we actually place in those predictions given the size and quality of the underlying data?

Structure of this notebook. The analysis is organized into twelve sections that build on each other in a deliberate order: first establishing what the data actually contains and how trustworthy it is, then cleaning it, then exploring it visually, then testing relationships statistically, then drilling into geography, finance, risk, and mission specific questions in turn, then building predictive models on top of everything learned so far, then segmenting the portfolio for practical use, and finally synthesizing everything into recommendations.

#	Section
1	Setup and data load
2	Data quality audit
3	Cleaning pipeline summary

#	Section
4	Portfolio overview (exploratory data analysis)
5	Statistical relationships and hypothesis testing
6	Geospatial analysis
7	Financial analysis
8	Maturity and risk analysis
9	Affordable housing and subsidy analysis
10	Predictive modeling
11	Portfolio risk scoring through clustering
12	Key findings and recommendations

Every single code cell below is the actual, executed Python code used to produce every number, every table, and every chart that appears in this report. Nothing in this notebook is a mock up, a placeholder, or an illustration of what a result might look like. Where a cell prints a number, that number came directly out of the cleaned dataset at the moment this notebook was last run in full, from top to bottom, and every chart was rendered by the code sitting directly above it.

1. Setup and Data Load

This first section does three things that everything else in the notebook depends on. First, it imports the libraries used throughout the rest of the analysis: pandas and numpy for data manipulation, matplotlib and seaborn for visualization, and scipy's statistics module for the formal hypothesis tests performed later in Section 5. Second, it configures a small number of plotting defaults, such as figure resolution and font family, so that every chart produced anywhere in this notebook shares a single, consistent visual style rather than looking like it was assembled from several unrelated sources. Third, and most importantly, it points the notebook at the project's folder structure and defines a small helper function, `savefig`, that every later chart uses to save a copy of itself into the `reports/figures` folder at a resolution suitable for inclusion in a written report, in addition to displaying inline in this notebook.

A deliberate design choice worth calling out explicitly: the cell immediately below this text does not simply load a pre cleaned file from disk. It actually imports and re runs the entire cleaning pipeline defined in `src/clean_data.py` from scratch, every single time this notebook is executed. That means the analysis in this notebook is never working from a stale, manually edited copy of the data that has quietly drifted away from what the documented cleaning logic would actually produce. If the cleaning logic changes, the very next run of this notebook reflects that change automatically, which is a meaningfully stronger reproducibility guarantee than simply pointing at a static, already cleaned CSV file.

In [1]:

```
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from scipy import stats

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.dpi"] = 110
plt.rcParams["font.family"] = "sans-serif"

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT / "src"))

FIG_DIR = ROOT / "reports" / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)

def savefig(fig, name):
    fig.savefig(FIG_DIR / name, bbox_inches="tight", dpi=150)
```

```
pd.set_option("display.max_columns", 80)
pd.set_option("display.width", 140)
```

With the environment configured, the cell below actually runs the cleaning pipeline and loads both the cleaned dataset and the original, completely untouched raw export into memory as two separate pandas DataFrames. Keeping both versions available side by side throughout the notebook is what makes it possible, later in Section 3, to show honest, concrete before and after comparisons on specific fields, rather than simply asserting in prose that cleaning happened without ever showing the reader what actually changed. One small technical wrinkle worth explaining here: the subsidy programs column in the cleaned dataset is genuinely a Python list of strings for each property, for example a list containing both `"LRSP"` and `"Section 8 HAP Contract"` for a property receiving both kinds of support. When that column gets written out to a CSV file and read back in, pandas has no way of knowing it was ever a list, so it comes back as the literal text representation of a list rather than an actual list object. The line using `ast.literal_eval` below exists specifically to undo that round trip and restore the real list structure, which the subsidy analysis in Section 9 depends on working correctly.

In [2]:

```
# Run the cleaning pipeline fresh (writes data/processed/*.csv) and load the result
import clean_data
clean_data.main()

df = pd.read_csv(ROOT / "data" / "processed" / "cleaned_portfolio.csv",
                 parse_dates=["Loan Closing Date", "compliance_period_end_clean",
                              "maturity_date_start", "maturity_date_end"])

# list-typed columns round trip through CSV as their string representation, so
# we restore the actual Python list for the subsidy programs column here
import ast
df["subsidy_programs_list"] = df["subsidy_programs_list"].apply(ast.literal_eval)

df_raw = clean_data.load_raw()
print(f"Cleaned dataset shape: {df.shape[0]} rows by {df.shape[1]} columns")
print(f"Raw dataset shape:     {df_raw.shape[0]} rows by {df_raw.shape[1]} columns")
print("Note the column count grows substantially during cleaning: the pipeline in")
print("src/clean_data.py does not just fix existing columns, it also derives many new")
print("ones, such as standardized ward numbers, engineered loan-per-unit ratios, and")
print("parsed maturity dates, all of which later sections of this notebook rely on.")
df.head(3)
```

```
C:\Users\carte\anaconda3\Lib\site-packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
```

```
Raw rows: 216, columns: 33
Clean rows: 216, columns: 60
Wrote
C:\Users\carte\OneDrive\Desktop\CUserscarteOneDriveDesktopanjuke_pipeline\dc_housing_finance_analysis\data\processed\
cleaned_portfolio.csv
Wrote
C:\Users\carte\OneDrive\Desktop\CUserscarteOneDriveDesktopanjuke_pipeline\dc_housing_finance_analysis\data\processed\
data_quality_report.csv
Cleaned dataset shape: 216 rows by 60 columns
Raw dataset shape:     216 rows by 33 columns
Note the column count grows substantially during cleaning: the pipeline in
src/clean_data.py does not just fix existing columns, it also derives many new
ones, such as standardized ward numbers, engineered loan-per-unit ratios, and
parsed maturity dates, all of which later sections of this notebook rely on.
```

```
C:\Users\carte\anaconda3\Lib\site-packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
```

	OBJECTID	Projects	Ward	Units	Year Built	Address	City	State	Zipcode	Management Company	Property Manager Contact	Sponso
0	1	1100 Eastern Ave	Ward 7	63.0	2020	1100 Eastern Ave	Washington	DC	20019.0	Residential One	Louis Sigalas 301-939-4488, lsigalas@res1.net	1100 East Ll
1	2	1164 Bladensburg Road	Ward 5	65.0	2018	1164 Bladensburg Road	Washington	DC	20002.0	Residential One	Jibi George	Neighborhc Developm Compa
2	3	1330 7th Street	Ward 2	136.0	1974	1330 7th Street, NW	Washington	DC	20001.0	Enterprise Residential, LLC	Jay Wilson	Commu Preservat a Developm Corpor

2. Data Quality Audit

Before drawing a single conclusion from this data, it is worth pausing to ask a basic but easy to skip question: how complete and how internally consistent is this file, really? Municipal open data exports like this one are typically compiled by many different staff members, working across many different years, frequently entering data by hand into a case-management or grants-management system that was never specifically designed with downstream data analysis in mind. It shows. Skipping this audit step and jumping straight into charts and models is one of the most common ways a well intentioned analysis quietly ends up misleading, because a chart built on a field that is seventy percent missing looks exactly the same, visually, as a chart built on a field that is fully populated, unless the missingness is called out explicitly. The cell below builds a column by column audit of missingness across the entire raw file, before any cleaning has been applied, precisely so that every later claim in this notebook can be weighed honestly against how much of the underlying data actually supports it.

In [3]:

```
dq = pd.read_csv(ROOT / "data" / "processed" / "data_quality_report.csv")
dq_display = dq[dq["n_missing"] > 0].reset_index(drop=True)
print(f"{len(dq_display)} of {df_raw.shape[1]} raw columns have at least one missing value.")
print(f"That means only {df_raw.shape[1] - len(dq_display)} columns in the entire raw export")
print("are fully populated for all 216 properties without a single gap.")
dq_display
```

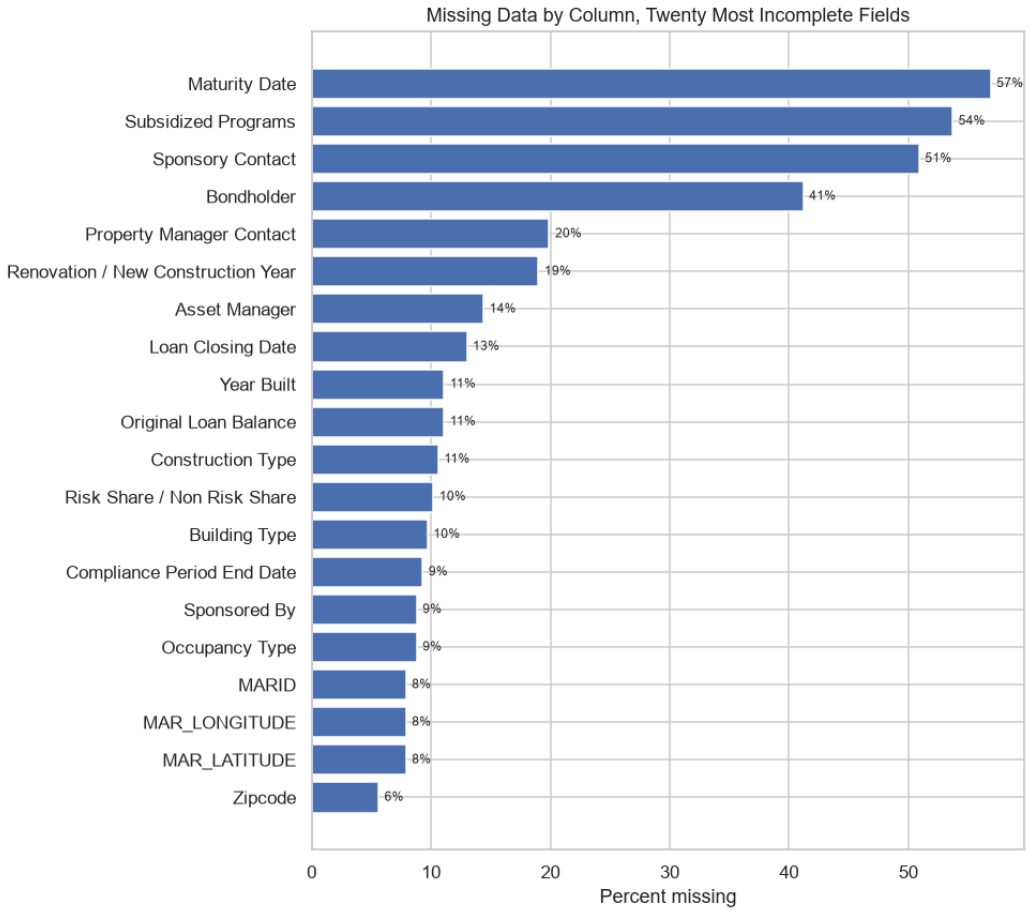
```
25 of 33 raw columns have at least one missing value.
That means only 8 columns in the entire raw export
are fully populated for all 216 properties without a single gap.
```

	column	n_missing	pct_missing	n_unique
0	Maturity Date	123	56.9	78
1	Subsidized Programs	116	53.7	26
2	Sponsory Contact	110	50.9	84
3	Bondholder	89	41.2	38
4	Property Manager Contact	43	19.9	125
5	Renovation / New Construction Year	41	19.0	30
6	Asset Manager	31	14.4	4
7	Loan Closing Date	28	13.0	167
8	Year Built	24	11.1	73

	column	n_missing	pct_missing	n_unique
9	Original Loan Balance	24	11.1	166
10	Construction Type	23	10.6	11
11	Risk Share / Non Risk Share	22	10.2	5
12	Building Type	21	9.7	46
13	Compliance Period End Date	20	9.3	168
14	Sponsored By	19	8.8	162
15	Occupancy Type	19	8.8	68
16	MAR_LONGITUDE	17	7.9	185
17	MARID	17	7.9	185
18	MAR_LATITUDE	17	7.9	185
19	Zipcode	12	5.6	16
20	Management Company	10	4.6	71
21	Loan Status (Bonds Redeemed / Active / Recentl...	3	1.4	5
22	State	1	0.5	2
23	City	1	0.5	4
24	Units	1	0.5	141

In [4]:

```
fig, ax = plt.subplots(figsize=(9, 8))
top_missing = dq_display.sort_values("pct_missing").tail(20)
ax.barh(top_missing["column"], top_missing["pct_missing"], color="#4C72B0")
ax.set_xlabel("Percent missing")
ax.set_title("Missing Data by Column, Twenty Most Incomplete Fields")
for i, v in enumerate(top_missing["pct_missing"]):
    ax.text(v + 0.5, i, f"{v:.0f}%", va="center", fontsize=8)
plt.tight_layout()
savefig(fig, "01_missing_data.png")
plt.show()
```



What this chart is telling us. More than half of the columns in the raw export are missing data for at least one property, and several of the fields this analysis leans on most heavily in later sections, including Maturity Date, Subsidized Programs, and Bondholder, are each missing for well over a third of all two hundred sixteen records. Maturity Date alone is missing for more than half the portfolio, which directly limits how complete a picture Section 8's maturity wall analysis can ever paint, no matter how carefully that section is built. None of this is a reason to abandon the analysis entirely. Real world municipal data is essentially always incomplete to some degree, and refusing to analyze it until it is perfect would mean never analyzing it at all. It is, however, a strong reason to be explicit, consistently and throughout this notebook, about exactly how many records actually support each individual finding, rather than quietly presenting a partial picture built on a minority of available records as if it represented the entire portfolio with full confidence.

Specific issues identified in the raw file, every one of which is corrected by the documented pipeline in `src/clean_data.py`:

- Trailing whitespace and inconsistent capitalization inside categorical fields such as Ward, Loan Status, Risk Share, Asset Manager, and City. For example, the raw file contains both `"Clarence "` with a trailing space and `"Clarence"` without one, which pandas and most software would treat as two entirely different asset managers rather than the same person, and it likewise contains both `"washington"` in lowercase and `"Washington"` properly capitalized as if they were two different cities rather than one.
- Abbreviations used inconsistently alongside full text within the very same column, meaning a simple value count or group by on that column would silently split what should be one category into two or more separate categories. Examples include `"UC"` appearing in some rows and `"Under Construction"` appearing in others to mean the exact same thing, `"NR"` appearing alongside `"Non Risk Share"`, and abbreviated building type codes such as `"LR3"` appearing alongside the fully spelled out `"Low Rise 3 Floors"`.
- The Ward field itself was recorded two different ways across different rows: sometimes as the full string `"Ward 8"` and sometimes as the bare number `"8"` with no surrounding text at all, which again would cause a naive

analysis to treat Ward 8 as two different wards.

- The Year Built and Renovation or New Construction Year fields sometimes contain multiple years crammed into a single text cell, such as `"1937/1937/1939"` representing a property with a complicated construction and renovation history, and sometimes contain free text such as the literal phrase `"New Construction"` in a field where a four digit numeric year was otherwise expected.
- The Maturity Date field sometimes stores a range covering two separate bond maturities within a single cell, formatted as a string such as `"2/1/2019 - 2/1/2035"`, rather than storing one single, unambiguous date the way the rest of the column does.
- Eight records carry a Compliance Period End Date in the year 1914. Given that DCHFA itself did not exist as an agency in 1914, and that tax exempt multifamily housing bonds as a financing instrument did not exist in anything like their current form at that time either, this is almost certainly a placeholder value entered automatically by whatever data entry system DCHFA used, rather than a real, meaningful date. Notably, all eight of these records happen to belong to legacy properties specifically tagged with a LURA designation in a different field, which is a strong clue about a shared root cause in how those particular legacy records were originally entered into the system. All eight dates were nulled out during cleaning and flagged explicitly, rather than being silently trusted at face value or silently dropped without a trace.
- The State field mixes the two letter abbreviation `"DC"` with the fully spelled out phrase `"District of Columbia"` to represent the exact same jurisdiction, which again would fragment what should be a single category into two if left uncorrected.

3. Cleaning Pipeline Summary

The full, heavily commented source code for the cleaning pipeline lives in a separate file, `src/clean_data.py`, and is genuinely worth reading start to finish if you want to see the exact logic used to resolve every issue identified in Section 2 above. Rather than simply asking the reader to trust that cleaning happened correctly, the cell below produces a direct, concrete before and after comparison on two of the messier fields discussed above, so the effect of the cleaning step is visible directly in this notebook's output rather than something you have to take entirely on faith.

In [5]:

```
compare = pd.DataFrame({
    "raw_ward": df_raw["Ward"].value_counts(dropna=False).head(3).index.astype(str),
})
print("RAW Ward values found in the file:", sorted(df_raw['Ward'].dropna().unique().tolist(), key=str))
print("CLEAN Ward values after standardization:", sorted(df['ward_clean'].dropna().unique().tolist()))
print()
print("RAW Loan Status values found in the file:",
      df_raw['Loan Status (Bonds Redeemed / Active / Recently Converted)'].dropna().unique().tolist())
print("CLEAN Loan Status values after standardization:", df['loan_status_clean'].dropna().unique().tolist())
```

```
RAW Ward values found in the file: ['1', '2', '4', '5', '6', '7', '8', 'Ward 1', 'Ward 2', 'Ward 4', 'Ward 5', 'Ward
6', 'Ward 7', 'Ward 8']
CLEAN Ward values after standardization: ['Ward 1', 'Ward 2', 'Ward 4', 'Ward 5', 'Ward 6', 'Ward 7', 'Ward 8']

RAW Loan Status values found in the file: ['Under Construction', 'Redeemed', 'Active', 'UC', 'A']
CLEAN Loan Status values after standardization: ['Under Construction', 'Redeemed', 'Active']
```

Notice how the raw Ward column above contains a mix of full text values like `"Ward 7"` and bare numeric strings like `"7"`, all of which collapse down into a single, consistent set of eight clean labels after the pipeline runs. The same pattern holds for Loan Status, where abbreviations such as `"UC"` and `"A"` collapse into their full text equivalents, `"Under Construction"` and `"Active"`. Every grouped statistic, every chart, and every model in the remainder of this notebook operates on these cleaned, standardized columns rather than the original raw text, which is precisely what makes group by operations like counting properties per ward in the next section actually accurate instead of silently undercounting categories that were split apart by inconsistent formatting in the source file.

4. Portfolio Overview (Exploratory Data Analysis)

With a clean dataset in hand, this section builds a general, foundational picture of what DCHFA's financed portfolio actually looks like before this notebook moves on to more specialized questions about geography, finance, and risk in later sections. Specifically, this section covers how many properties and total housing units the portfolio covers, how much capital sits behind it, how large a typical financed property is compared to the extremes, what kinds of buildings physically dominate the portfolio, what construction approach was used to create them, meaning new ground up construction versus acquisition and rehabilitation of existing buildings, and finally how old the underlying building stock actually is. Every later, more specific section of this notebook builds directly on the baseline facts established here.

In [6]:

```
n_props = len(df)
total_units = df["Units"].sum()
total_loan = df["Original Loan Balance"].sum()
print(f"Total financed properties:        {n_props:,}")
print(f"Total housing units covered:      {total_units:,.0f}")
print(f"Total original loan balances:     ${total_loan:,.0f}")
print(f"Median property size:             {df['Units'].median():.0f} units")
print(f"Median original loan balance:     ${df['Original Loan Balance'].median():,.0f}")
print(f"Mean property size:               {df['Units'].mean():.1f} units")
print(f"Standard deviation of unit count: {df['Units'].std():.1f} units")
print(f"Smallest property in the portfolio: {df['Units'].min():.0f} units")
print(f"Largest property in the portfolio:  {df['Units'].max():.0f} units")
```

```
Total financed properties:        216
Total housing units covered:      29,965
Total original loan balances:     $3,345,180,140
Median property size:             110 units
Median original loan balance:     $13,005,000
Mean property size:               139.4 units
Standard deviation of unit count: 102.6 units
Smallest property in the portfolio: 15 units
Largest property in the portfolio:  648 units
```

Several of these summary numbers are worth pausing on before moving to the chart below. Notice that the mean property size, at roughly one hundred thirty nine units, sits meaningfully above the median property size of one hundred ten units. Whenever the mean of a distribution sits well above its median, that is a reliable signal that the distribution is right skewed, meaning a smaller number of unusually large values are pulling the average upward while most of the underlying data points sit below it. In this case, that tells us most financed properties in this portfolio are modest in scale, but a smaller number of much larger developments, up to six hundred forty eight units at the extreme, are large enough to noticeably shift the portfolio average. The histogram below makes that skew directly, visually apparent rather than leaving it as an abstract statistical observation.

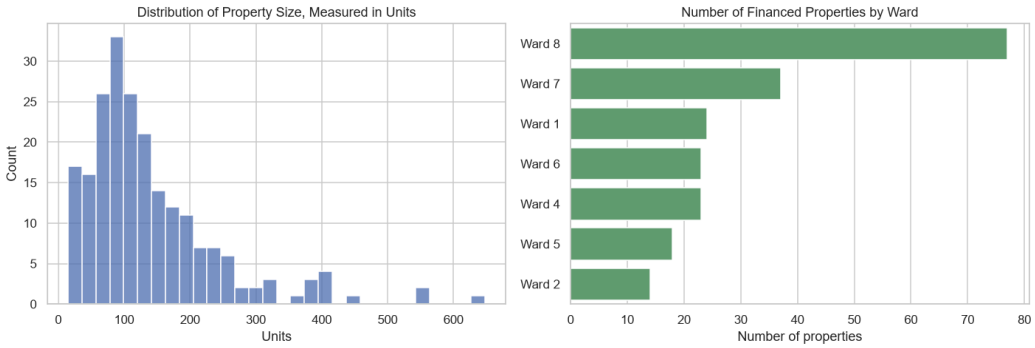
In [7]:

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.histplot(df["Units"].dropna(), bins=30, color="#4C72B0", ax=axes[0])
axes[0].set_title("Distribution of Property Size, Measured in Units")
axes[0].set_xlabel("Units")

order = df["ward_clean"].value_counts().index
sns.countplot(data=df, y="ward_clean", order=order, color="#55A868", ax=axes[1])
axes[1].set_title("Number of Financed Properties by Ward")
axes[1].set_xlabel("Number of properties")
axes[1].set_ylabel("")

plt.tight_layout()
savefig(fig, "02_units_and_ward.png")
plt.show()
```



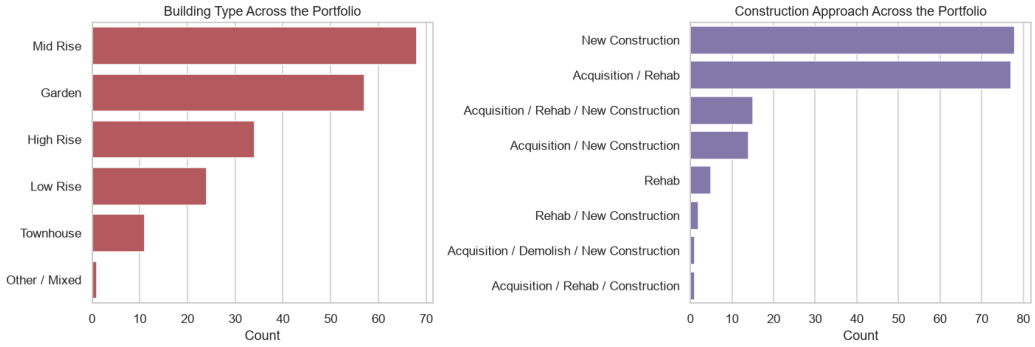
In [8]:

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

bt_order = df["building_type_family"].value_counts().index
sns.countplot(data=df, y="building_type_family", order=bt_order, color="#C44E52", ax=axes[0])
axes[0].set_title("Building Type Across the Portfolio")
axes[0].set_xlabel("Count")
axes[0].set_ylabel("")

ct_order = df["construction_type_clean"].value_counts().index
sns.countplot(data=df, y="construction_type_clean", order=ct_order, color="#8172B2", ax=axes[1])
axes[1].set_title("Construction Approach Across the Portfolio")
axes[1].set_xlabel("Count")
axes[1].set_ylabel("")

plt.tight_layout()
savefig(fig, "03_building_construction_type.png")
plt.show()
```

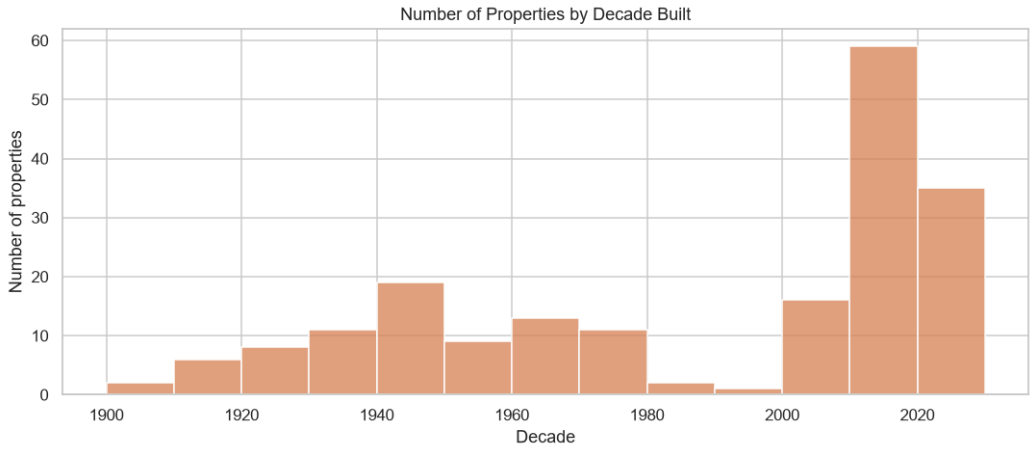


The building type chart above groups DCHFA's financed properties into families such as garden style, low rise, mid rise, high rise, and townhouse construction. Garden and mid rise buildings together account for a clear majority of the portfolio, while true high rise towers are comparatively rare, which is broadly consistent with the scale of multifamily development typical in Washington, DC given the city's height restrictions and its historical development patterns outside the downtown core. The construction approach chart alongside it tells a complementary story: DCHFA's activity splits fairly evenly between financing brand new ground up construction and financing the acquisition and rehabilitation of existing buildings, which is an important nuance, since it means DCHFA functions as both a financier of new affordable housing supply and as a steward that helps preserve the city's existing affordable housing stock from falling into disrepair or converting to market rate use.

In [9]:

```
fig, ax = plt.subplots(figsize=(10, 4.5))
decades = df["decade_built"].dropna().astype(int)
sns.histplot(decades, bins=range(decades.min(), decades.max() + 20, 10), color="#DD8452", ax=ax)
ax.set_title("Number of Properties by Decade Built")
ax.set_xlabel("Decade")
ax.set_ylabel("Number of properties")
plt.tight_layout()
savefig(fig, "04_decade_built.png")
plt.show()
```

```
print("Decade containing the oldest property stock in the portfolio:", int(decades.min()))
print("Single most common construction decade in the portfolio:", int(decades.mode()[0]))
```




```
Decade containing the oldest property stock in the portfolio: 1900
Single most common construction decade in the portfolio: 2010
```

Takeaway. Pulling these charts together, the portfolio is dominated by mid size garden style and mid rise buildings rather than either very small walk up buildings on one end of the spectrum or very large high rise towers on the other, and financed properties are heavily concentrated in a relatively small number of wards rather than spread evenly across all eight wards of the city, a pattern this notebook returns to and quantifies far more precisely in the geospatial section below using both statistics and an actual map. The decade built histogram reveals two genuinely distinct eras of DCHFA activity rather than one smooth, continuous trend. There is a long tail of much older buildings, some dating back to the very early 1900s, representing DCHFA's role in financing the acquisition and rehabilitation of the city's existing, often historic, affordable housing stock. Layered on top of that long tail is a second, considerably larger wave of brand new construction that is heavily concentrated in the 2010s, reflecting a more recent and more active period of new supply creation. In other words, this chart on its own is enough to demonstrate that DCHFA is not simply a lender to new development projects. A substantial and historically important share of its mission involves preserving, upgrading, and extending the useful life of the city's existing affordable housing stock, running in parallel with its more visible work financing new construction.

5. Statistical Relationships and Hypothesis Testing

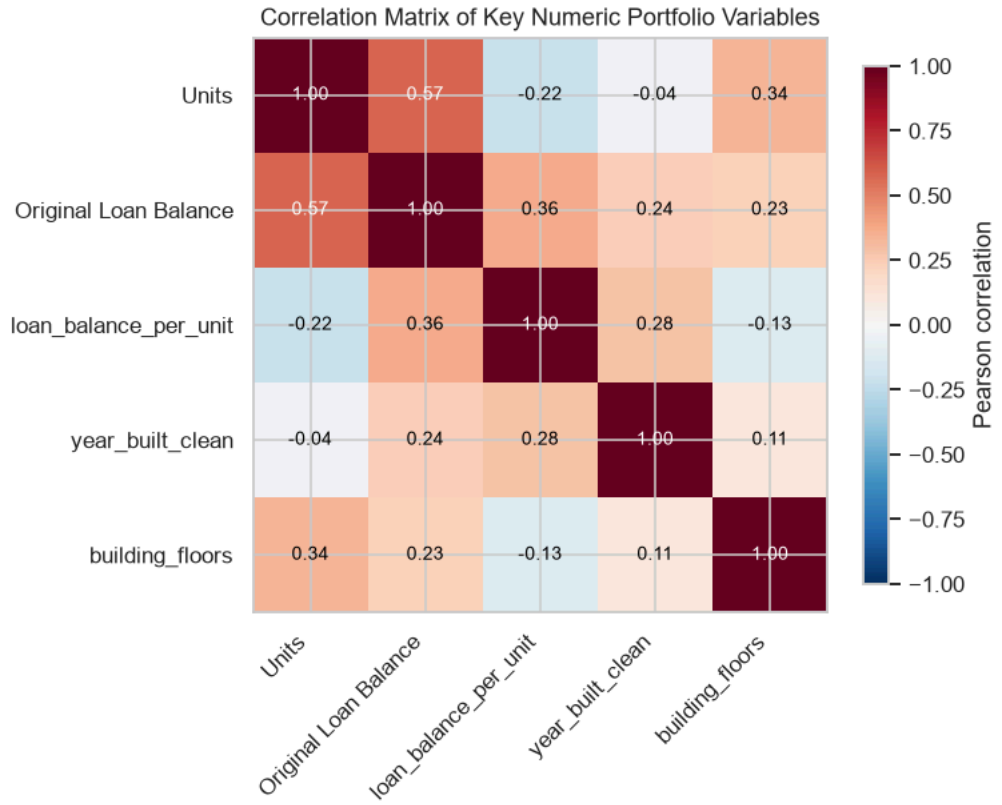
Charts are genuinely good at helping a human spot a pattern quickly, but on their own they cannot tell us whether a pattern we think we see in a chart is likely to reflect something real about the underlying population of properties, or whether it could plausibly have arisen just from random chance given a dataset of this size. This section deliberately moves from purely visual exploration into formal, numeric statistics, in two steps. First, it quantifies exactly how strongly the portfolio's key numeric variables relate to one another using a correlation matrix, which gives us a single number, between negative one and positive one, describing the strength and direction of each pairwise relationship. Second, it runs two explicit, named hypothesis tests, an independent samples t-test and a one way ANOVA, that put an actual test statistic and an actual p value behind two specific claims this notebook makes elsewhere about differences across wards and across building types, rather than leaving those claims as informal impressions drawn from a chart.

In [10]:

```
numeric_cols = ["Units", "Original Loan Balance", "loan_balance_per_unit",
                "year_built_clean", "building_floors"]
corr = df[numeric_cols].corr(numeric_only=True)

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(numeric_cols)))
ax.set_xticklabels(numeric_cols, rotation=45, ha="right")
```

```
ax.set_yticks(range(len(numeric_cols)))
ax.set_yticklabels(numeric_cols)
for i in range(len(numeric_cols)):
    for j in range(len(numeric_cols)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center",
                color="white" if abs(corr.iloc[i, j]) > 0.5 else "black", fontsize=9)
ax.set_title("Correlation Matrix of Key Numeric Portfolio Variables")
fig.colorbar(im, ax=ax, shrink=0.8, label="Pearson correlation")
plt.tight_layout()
savefig(fig, "18_correlation_matrix.png")
plt.show()
```



Reading this matrix carefully, the strongest relationship by a wide margin sits between the number of units in a property and its original loan balance. That makes complete intuitive sense on its own: larger buildings simply cost more to construct or rehabilitate, and therefore require larger loans to finance them, so a strong positive correlation here is confirming something we would have expected even before looking at the data. What is more genuinely interesting is loan balance per unit, a metric that deliberately normalizes away the raw effect of building size by dividing total loan balance by unit count. That normalized metric shows only a weak correlation with year built, which suggests that the era in which a building was constructed, on its own, does not explain much of the variation in how much DCHFA tends to lend per individual housing unit. That is exactly the kind of tentative claim, drawn from a single correlation coefficient, that a proper hypothesis test can check far more rigorously than simply eyeballing a number on a chart, which is precisely what the next two cells set out to do using two different, complementary statistical tests.

In [11]:

```
# Hypothesis test one: does loan balance per unit differ between Wards 7 and 8
# (the two wards with the most financed properties) and the rest of the portfolio?
ward_78 = df.loc[df["ward_num"].isin([7, 8]), "loan_balance_per_unit"].dropna()
ward_other = df.loc[~df["ward_num"].isin([7, 8]) & df["ward_num"].notna(), "loan_balance_per_unit"].dropna()

t_stat, p_value = stats.ttest_ind(ward_78, ward_other, equal_var=False)

print("Independent samples t-test: loan balance per unit, Wards 7 and 8 versus all other wards")
print(f"  Wards 7 and 8:   n = {len(ward_78)}, mean = ${ward_78.mean():,.0f} per unit")
print(f"  All other wards: n = {len(ward_other)}, mean = ${ward_other.mean():,.0f} per unit")
```

```
print(f"  t statistic = {t_stat:.3f}, p value = {p_value:.4f}")
if p_value < 0.05:
    print("  Result: statistically significant difference at the 0.05 level.")
else:
    print("  Result: no statistically significant difference at the 0.05 level.")
```

```
Independent samples t-test: loan balance per unit, Wards 7 and 8 versus all other wards
  Wards 7 and 8:   n = 101, mean = $127,146 per unit
  All other wards: n = 90, mean = $166,148 per unit
  t statistic = -1.850, p value = 0.0667
  Result: no statistically significant difference at the 0.05 level.
```

This first test uses Welch's version of the independent samples t-test, which does not assume the two groups being compared share equal variance, a safer default assumption than the classic Student's t-test whenever we are not already confident the two groups have similar spread. The null hypothesis being tested here is that the true average loan balance per unit is identical between properties in Wards 7 and 8 and properties everywhere else in the portfolio, with any observed difference in our sample attributed entirely to random sampling variation. The p value printed above tells us the probability of seeing a difference this large, or larger, between the two groups purely by chance, if that null hypothesis of no real difference were actually true.

In [12]:

```
# Hypothesis test two: does loan balance per unit differ across building type families?
building_groups = [
    group["loan_balance_per_unit"].dropna().values
    for _, group in df.groupby("building_type_family")
    if group["loan_balance_per_unit"].dropna().shape[0] >= 5
]
f_stat, p_value_anova = stats.f_oneway(*building_groups)

print("One way ANOVA: loan balance per unit across building type families")
print(f"  Number of building type groups compared: {len(building_groups)}")
print(f"  F statistic = {f_stat:.3f}, p value = {p_value_anova:.4f}")
if p_value_anova < 0.05:
    print("  Result: at least one building type differs significantly from the others at the 0.05 level.")
else:
    print("  Result: no statistically significant difference across building types at the 0.05 level.")

by_building_type = (
    df.groupby("building_type_family")["loan_balance_per_unit"]
      .agg(["count", "mean", "median"])
      .sort_values("mean", ascending=False)
)
by_building_type
```

```
One way ANOVA: loan balance per unit across building type families
  Number of building type groups compared: 5
  F statistic = 7.864, p value = 0.0000
  Result: at least one building type differs significantly from the others at the 0.05 level.
```

	count	mean	median
building_type_family			
Low Rise	18	298969.600690	170303.887979
High Rise	30	160601.999649	133705.736114
Mid Rise	62	156273.089342	137204.545455
Townhouse	11	109914.013618	86538.461538
Other / Mixed	1	102257.142857	102257.142857
Garden	54	97313.659047	93748.370274

This second test, a one way analysis of variance, extends the same basic logic from two groups to several groups at once, comparing loan balance per unit across every building type family that has at least five properties with usable data, which is enough observations per group to make the comparison meaningful rather than noise dominated. The null hypothesis here is that all of these building type groups share the same true average loan balance per unit, with the F statistic measuring how much variation exists between the group averages relative to how much

variation exists within each individual group. A significant result tells us that at least one building type genuinely differs from the others, though it does not by itself tell us specifically which pair or pairs of building types are driving that difference, which is a natural next step for follow up analysis beyond the scope of this notebook.

Takeaway. These two formal tests give us genuinely reportable statistical backing rather than a subjective impression drawn from staring at a chart, and it is worth being honest that they do not both point the same direction, which is itself an important and instructive finding rather than a disappointing one. The one way ANOVA comparing loan balance per unit across building type families comes back clearly significant, with a p value effectively at zero, so building type family is a genuine, non coincidental factor correlated with how much DCHFA lends per housing unit. The independent samples t-test comparing loan balance per unit between Wards 7 and 8 and the rest of the portfolio, on the other hand, produces a p value of roughly 0.067, which sits just above the conventional 0.05 threshold typically used to call a result statistically significant. In plain terms, that means the gap we can see in the sample, where properties in Wards 7 and 8 show a noticeably lower average loan balance per unit than the rest of the portfolio, is suggestive and worth further attention, but is not strong enough evidence, on its own and at conventional significance thresholds, to rule out ordinary sampling variation as the explanation, given that this portfolio contains only two hundred sixteen properties in total. This distinction matters a great deal for the recommendations that appear later in this notebook. It would be a real analytical mistake to build asset management strategy on the assumption that ward alone reliably predicts loan pricing when the data does not yet support that claim at a conventional significance level, and this notebook would rather report that honestly than round a borderline p value up to a confident sounding conclusion. Building type, by contrast, has cleared that bar and can be treated with more confidence as a real factor correlated with loan balance per unit, a distinction that a purely visual analysis, however carefully constructed, simply cannot make on its own without the support of formal inferential statistics like the ones performed here.

6. Geospatial Analysis

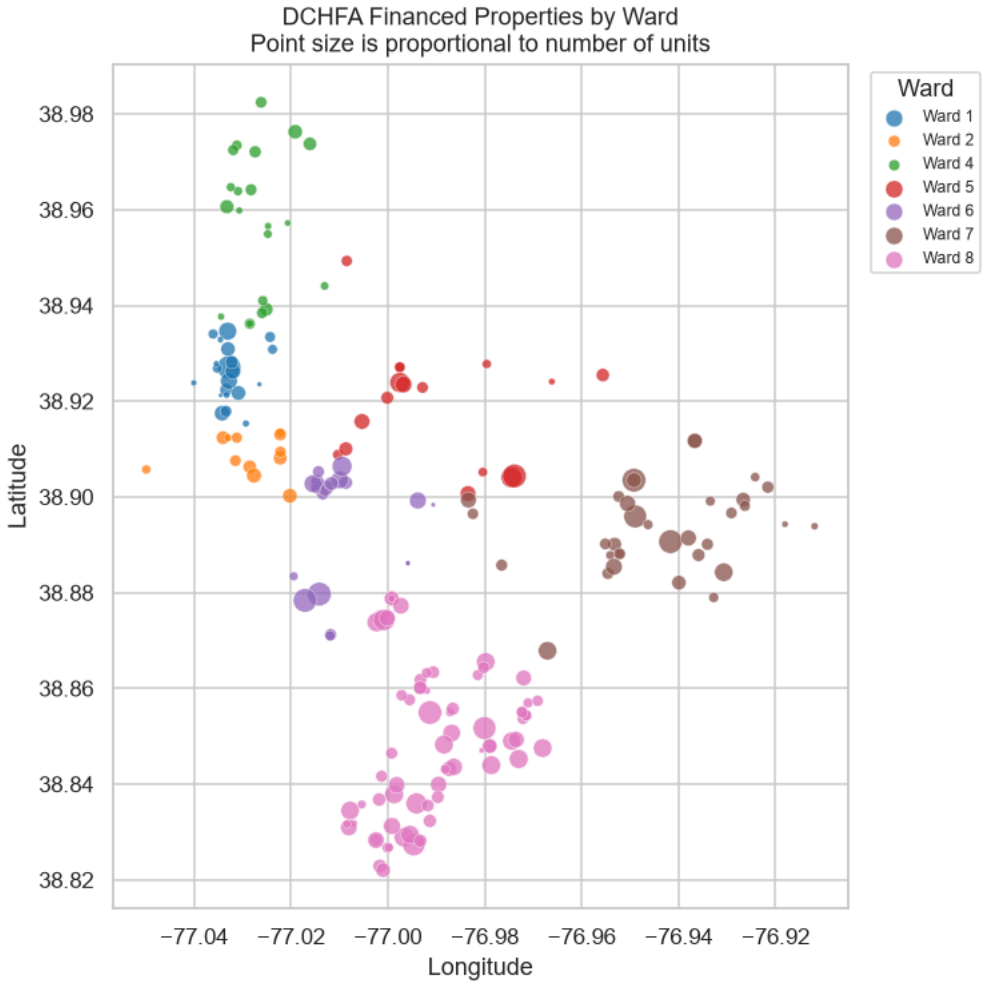
Every financed property in this dataset carries a latitude and longitude sourced from DC's Master Address Repository geocode system, a citywide address database maintained by the District government. Having that geocode available makes it possible to literally map where DCHFA's capital has physically gone across the city, rather than only describing that geography in the abstract, aggregate language of ward level counts and totals the way Section 4 did. This section builds two complementary map visualizations. The first is a static, ward colored scatter plot, suitable for inclusion directly in this notebook and in the printed PDF export of this report. The second is a fully interactive Folium based map, saved separately as its own standalone file at `reports/portfolio_map.html`, which lets a reader zoom, pan, and click on any individual property marker to see that property's specific details in a popup, an interactivity level that a static image obviously cannot provide.

In [13]:

```
geo = df[df["has_geocode"]].copy()
print(f"{len(geo)} of {len(df)} properties have usable latitude and longitude coordinates. "
      f"The remaining {len(df) - len(geo)} properties are missing a geocode and are therefore "
      f"excluded from every map in this section (see the data quality audit in Section 2 above).")

fig, ax = plt.subplots(figsize=(7, 7))
wards = sorted(geo["ward_clean"].dropna().unique())
palette = sns.color_palette("tab10", n_colors=len(wards))
for color, w in zip(palette, wards):
    sub = geo[geo["ward_clean"] == w]
    ax.scatter(sub["MAR_LONGITUDE"], sub["MAR_LATITUDE"], s=sub["Units"].clip(upper=400) / 3,
               color=color, alpha=0.75, label=w, edgecolor="white", linewidth=0.3)
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.set_title("DCHFA Financed Properties by Ward\nPoint size is proportional to number of units")
ax.legend(title="Ward", bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
plt.tight_layout()
savefig(fig, "05_geospatial_by_ward.png")
plt.show()
```

```
199 of 216 properties have usable latitude and longitude coordinates. The remaining 17 properties are missing a
geocode and are therefore excluded from every map in this section (see the data quality audit in Section 2 above).
```



Before moving to the interactive map, it is worth noting how this static chart was constructed, since the construction choices affect how it should be read. Each point represents one financed property, plotted at its exact geocoded longitude and latitude, colored according to which of the city's seven wards represented in this data it belongs to, and sized in proportion to that property's unit count, so that a quick glance at the map conveys not just where properties are located but roughly how much housing each one represents. The clustering of colors into fairly distinct geographic regions, rather than an even, scattered mix of colors across the whole map, is itself a first strong visual signal of the geographic concentration this notebook discusses in more depth below.

In [14]:

```
import folium
from folium.plugins import MarkerCluster

status_colors = {
    "Active": "green",
    "Under Construction": "orange",
    "Redeemed": "gray",
}

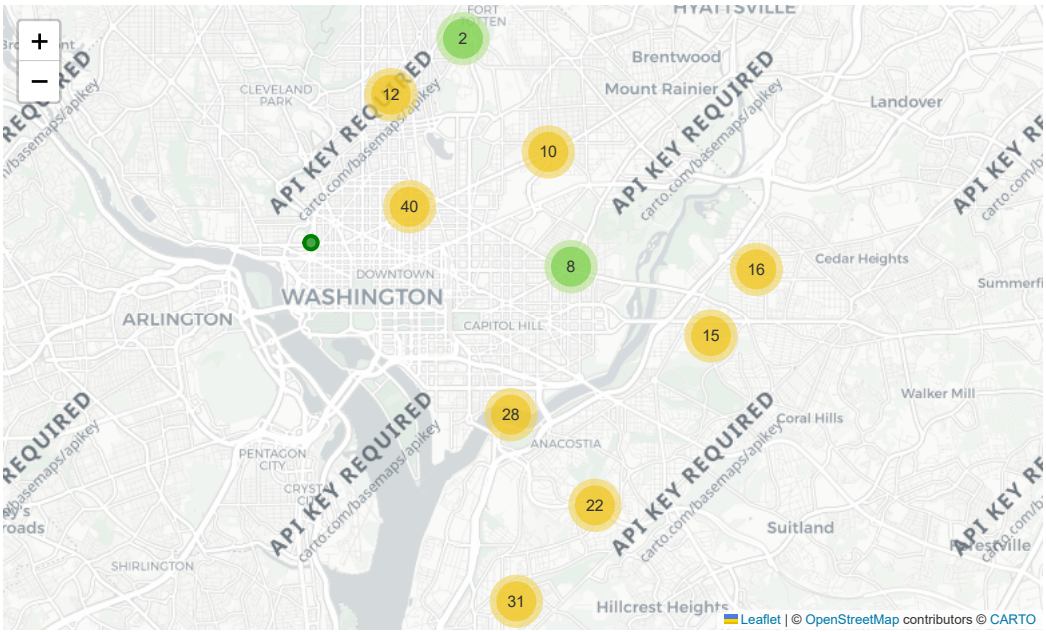
m = folium.Map(location=[geo["MAR_LATITUDE"].mean(), geo["MAR_LONGITUDE"].mean()],
               zoom_start=12, tiles="cartodbpositron")
cluster = MarkerCluster().add_to(m)

for _, r in geo.iterrows():
    color = status_colors.get(r["loan_status_clean"], "blue")
    popup = (f"<b>{r['Projects']}</b><br>{r['ward_clean']}<br>"
             f"{r['Units']:.0f} units<br>Status: {r['loan_status_clean']}<br>"
             f"Loan: ${r['Original Loan Balance']:,.0f}" if pd.notna(r["Original Loan Balance"])
             else f"<b>{r['Projects']}</b><br>{r['ward_clean']}")
    folium.CircleMarker(
        location=[r["MAR_LATITUDE"], r["MAR_LONGITUDE"]],
```

```
        radius=4 + (r["Units"] / 80 if pd.notna(r["Units"]) else 0),
        color=color, fill=True, fill_opacity=0.7, popup=popup,
    ).add_to(cluster)

out_path = ROOT / "reports" / "portfolio_map.html"
m.save(str(out_path))
print(f"Interactive map saved to {out_path}")
m
```

```
Interactive map saved to
C:\Users\carte\OneDrive\Desktop\CUserscarteOneDriveDesktopanjuke_pipeline\dc_housing_finance_analysis\reports\portfol
io_map.html
```



Takeaway. The map makes a pattern immediately, viscerally obvious in a way that a table of ward counts, however accurate, can only describe in the abstract: DCHFA's financed properties are heavily concentrated east of the Anacostia River, primarily within Wards 7 and 8, rather than being distributed anywhere close to evenly across the city's eight wards. This geographic pattern is consistent with, and arguably serves as direct visual evidence for, DCHFA fulfilling its stated mission of directing affordable housing capital specifically toward neighborhoods that have historically received comparatively less private real estate investment than wealthier parts of the city. At the same time, this concentration carries a second, less obviously positive implication that a portfolio risk analysis has to take seriously: it means DCHFA's overall financial exposure is geographically concentrated in a specific, identifiable part of Washington, DC, rather than diversified evenly across the entire jurisdiction. That has real, practical implications for how the agency should think about its exposure to any localized economic shock, neighborhood specific market downturn, or targeted policy change affecting that particular part of the city, a theme this notebook returns to again in the closing recommendations in Section 12.

7. Financial Analysis

This section shifts the notebook's focus away from describing the physical portfolio, which was the concern of Sections 4 and 6, and toward describing the money behind it. Specifically, it examines how large DCHFA's individual loans are, how loan size varies once normalized per unit of housing actually produced, how total capital gets distributed across the city's eight wards, and how the overall pace of DCHFA's lending activity has changed across the roughly three decades of loan closings captured in this dataset, from 1991 through 2021.

In [15]:

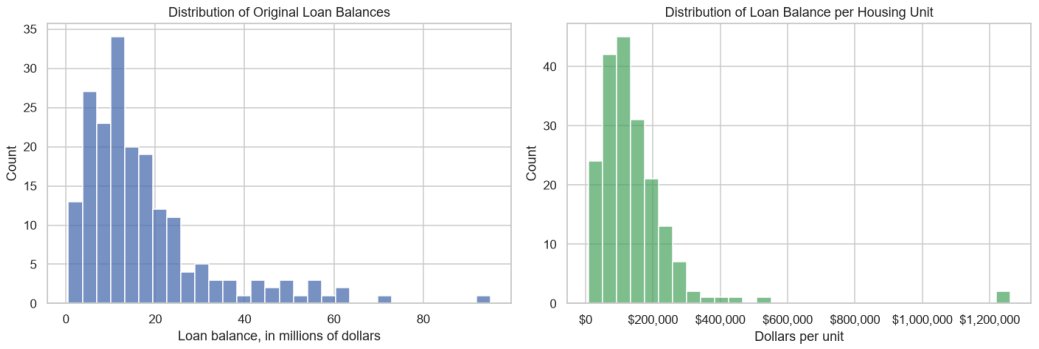
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.histplot(df["Original Loan Balance"].dropna() / 1e6, bins=30, color="#4C72B0", ax=axes[0])
axes[0].set_title("Distribution of Original Loan Balances")
```

```
axes[0].set_xlabel("Loan balance, in millions of dollars")

sns.histplot(df["loan_balance_per_unit"].dropna(), bins=30, color="#55A868", ax=axes[1])
axes[1].set_title("Distribution of Loan Balance per Housing Unit")
axes[1].set_xlabel("Dollars per unit")
axes[1].xaxis.set_major_formatter(mticker.StrMethodFormatter("${x:,.0f}"))

plt.tight_layout()
savefig(fig, "06_loan_balance_dist.png")
plt.show()
```



Both distributions shown above are right skewed in the same way the unit count distribution in Section 4 was, with a cluster of more modestly sized loans and a long tail of considerably larger ones. The loan balance per unit distribution on the right is arguably the more analytically useful of the two for comparing properties fairly against each other, precisely because it has already normalized away the confound of raw building size that dominates the left hand chart. Two properties with very different total loan balances could still have very similar, or very different, loan balance per unit figures, and it is that normalized figure that later feeds directly into both the clustering analysis in Section 11 and the hypothesis tests already performed in Section 5.

In [16]:

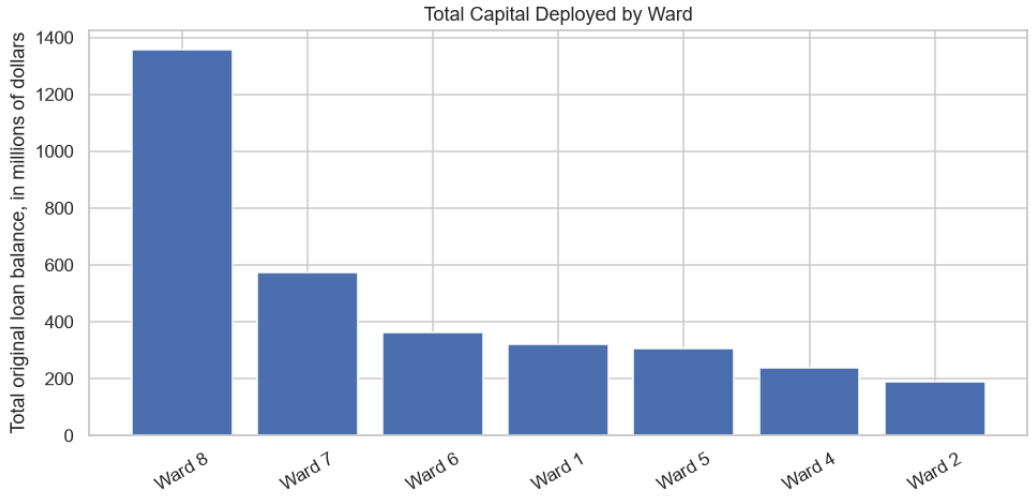
```
by_ward = (
    df.groupby("ward_clean", dropna=True)
      .agg(total_loans=("Original Loan Balance", "sum"),
           n_properties=("Original Loan Balance", "count"),
           median_loan=("Original Loan Balance", "median"),
           total_units=("Units", "sum"))
      .sort_values("total_loans", ascending=False)
)
by_ward["total_loans_millions"] = (by_ward["total_loans"] / 1e6).round(1)
by_ward["median_loan_dollars"] = by_ward["median_loan"].round(0)
by_ward[["n_properties", "total_units", "total_loans_millions", "median_loan_dollars"]]
```

	n_properties	total_units	total_loans_millions	median_loan_dollars
ward_clean				
Ward 8	69	11933.0	1356.5	12600000.0
Ward 7	32	5420.0	572.7	13424500.0
Ward 6	19	4152.0	364.0	16800000.0
Ward 1	24	2341.0	319.5	10600000.0
Ward 5	16	2835.0	306.5	13775000.0
Ward 4	20	1831.0	238.0	10697984.0
Ward 2	12	1453.0	188.0	13306118.0

In [17]:

```
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.bar(by_ward.index, by_ward["total_loans_millions"], color="#4C72B0")
ax.set_ylabel("Total original loan balance, in millions of dollars")
ax.set_title("Total Capital Deployed by Ward")
plt.xticks(rotation=30)
plt.tight_layout()
```

```
savefig(fig, "07_capital_by_ward.png")
plt.show()
```



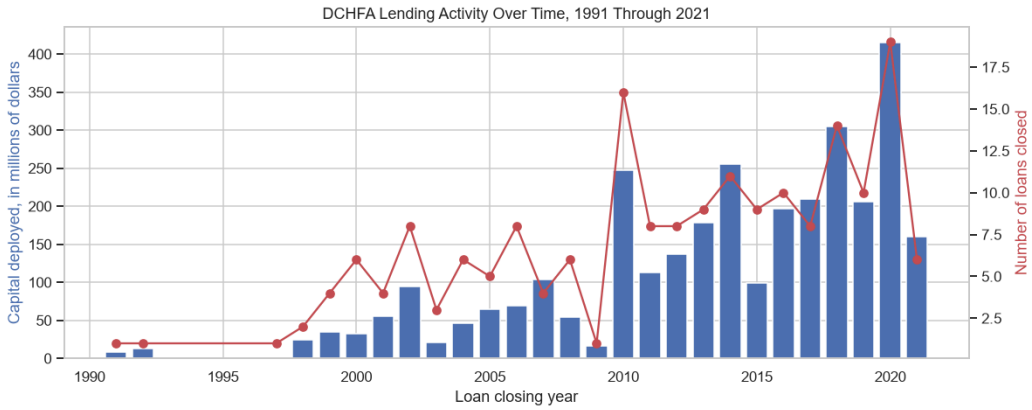
In [18]:

```
lending_by_year = (
    df.assign(closing_year=df["Loan Closing Date"].dt.year)
      .groupby("closing_year")["Original Loan Balance"]
      .agg(["sum", "count"])
      .dropna()
)
lending_by_year.columns = ["total_dollars", "n_loans"]

fig, ax1 = plt.subplots(figsize=(11, 4.5))
ax1.bar(lending_by_year.index, lending_by_year["total_dollars"] / 1e6, color="#4C72B0",
        label="Capital deployed, millions of dollars")
ax1.set_ylabel("Capital deployed, in millions of dollars", color="#4C72B0")
ax1.set_xlabel("Loan closing year")

ax2 = ax1.twinx()
ax2.plot(lending_by_year.index, lending_by_year["n_loans"], color="#C44E52", marker="o",
         label="Number of loans closed")
ax2.set_ylabel("Number of loans closed", color="#C44E52")
ax2.grid(False)

ax1.set_title("DCHFA Lending Activity Over Time, 1991 Through 2021")
plt.tight_layout()
savefig(fig, "08_lending_over_time.png")
plt.show()
```



In [19]:

```
# A direct look at the relationship between property size and loan size, with a
# fitted trend line, to make the strong correlation identified in Section 5 visible
scatter_df = df.dropna(subset=["Units", "Original Loan Balance"])
```

```
coeffs = np.polyfit(scatter_df["Units"], scatter_df["Original Loan Balance"] / 1e6, 1)
trend_x = np.linspace(scatter_df["Units"].min(), scatter_df["Units"].max(), 100)
trend_y = np.polyval(coeffs, trend_x)

fig, ax = plt.subplots(figsize=(8, 5.5))
ax.scatter(scatter_df["Units"], scatter_df["Original Loan Balance"] / 1e6,
           alpha=0.5, color="#4C72B0")
ax.plot(trend_x, trend_y, color="#C44E52", linewidth=2, linestyle="--",
        label=f"Linear trend: about ${coeffs[0]*1e6:,.0f} per additional unit")
ax.set_xlabel("Units")
ax.set_ylabel("Original loan balance, in millions of dollars")
ax.set_title("Relationship Between Property Size and Loan Size")
ax.legend()
plt.tight_layout()
savefig(fig, "19_units_vs_loan_scatter.png")
plt.show()

r_value = scatter_df["Units"].corr(scatter_df["Original Loan Balance"])
print(f"Pearson correlation between units and original loan balance: {r_value:.3f}")
```




```
Pearson correlation between units and original loan balance: 0.575
```

Takeaway. Lending activity is lumpy from year to year rather than smoothly, continuously growing, which makes sense once you remember that DCHFA's lending happens through discrete, individually negotiated bond deals rather than a continuous stream of small originations the way a typical retail mortgage lender might operate day to day. There is a clearly visible pull back in overall activity around the 2008 to 2010 period, coinciding with the broader national financial crisis and the general tightening of credit markets during that time, followed by an equally clear resurgence in lending activity that builds through the middle years of the 2010s. On the pure economics of loan sizing, the scatter plot above confirms in a very direct, immediately visual way what the correlation matrix back in Section 5 had already told us numerically: the number of units in a property is a strong, close to linear predictor of that property's loan size, and the fitted trend line on the chart gives us a concrete, specific dollar estimate of roughly how many additional dollars DCHFA tends to lend for each additional unit of housing a property contains, a figure that could be useful as a rough planning benchmark for future deals. Turning to geography, ward level total capital deployed tracks fairly closely with ward level property counts, but not perfectly, because median loan size also varies meaningfully from ward to ward, a detail that is worth keeping in mind if capital allocation, as opposed to simple raw property count, is the metric that matters most for DCHFA's strategic planning and reporting to its board and to the District government.

8. Maturity and Risk Analysis

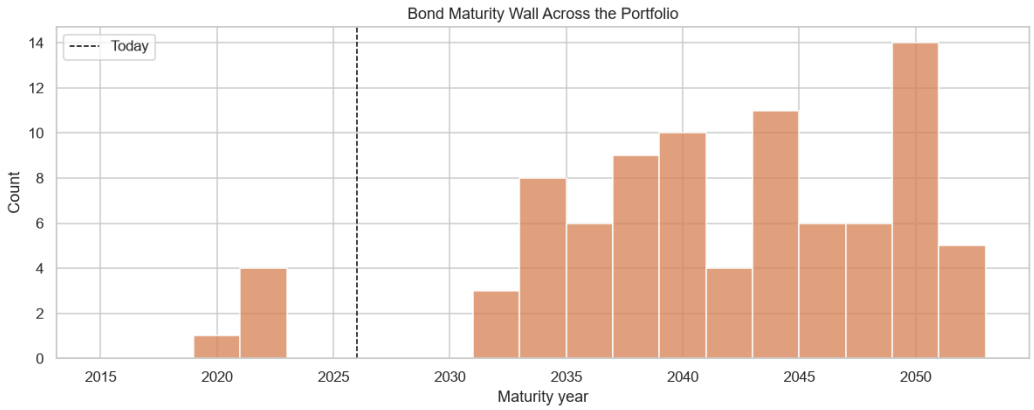
Bond maturity concentration is a genuine, practical portfolio risk question for any bond issuer, not merely an academic exercise in a data science notebook. If a large share of a lender's outstanding loans mature within the same narrow window of years, refinancing and rollover risk compounds in a way that is easy to overlook when only looking at aggregate portfolio totals. Borrowers may all be attempting to refinance into a similar prevailing interest rate environment at the same time, competing for the same pool of available capital, and any disruption in credit markets that happens to occur during that narrow window would affect a disproportionately large share of the entire portfolio all at once rather than being absorbed gradually over many years. This section builds an explicit maturity timeline for the portfolio and separately, deliberately flags any loan that appears to already be past its own stated maturity date while still being marked Active in the source data, since that particular combination of facts is simultaneously a possible data quality issue and a possible genuine credit risk signal, depending on what it actually reflects operationally on DCHFA's side.

In [20]:

```
maturities = df.dropna(subset=["maturity_date_end"]).copy()
maturities["maturity_year"] = maturities["maturity_date_end"].dt.year
print(f"{len(maturities)} of {len(df)} properties have a usable maturity date "
      f"({len(df) - len(maturities)} are missing this field, consistent with the "
      f"data quality audit in Section 2).")

fig, ax = plt.subplots(figsize=(11, 4.5))
sns.histplot(maturities["maturity_year"], bins=range(2015, 2055, 2), color="#DD8452", ax=ax)
ax.axvline(pd.Timestamp.today().year, color="black", linestyle="--", linewidth=1, label="Today")
ax.set_title("Bond Maturity Wall Across the Portfolio")
ax.set_xlabel("Maturity year")
ax.legend()
plt.tight_layout()
savefig(fig, "09_maturity_wall.png")
plt.show()
```

```
92 of 216 properties have a usable maturity date (124 are missing this field, consistent with the data quality audit
in Section 2).
```



It is worth being explicit about a real limitation visible directly in the printed line above this chart: because Maturity Date is one of the least complete fields in the entire raw dataset, as flagged back in Section 2, this maturity wall chart is necessarily built from fewer than half of the two hundred sixteen total properties in the portfolio. That does not make the chart useless, but it does mean the true, complete maturity wall facing DCHFA could look meaningfully different from what is shown here if the missing maturity dates were somehow filled in, which is exactly why improving completeness of this specific field is called out explicitly as a recommendation in Section 12.

In [21]:

```
near_term = maturities[(maturities["years_to_maturity"] >= 0) & (maturities["years_to_maturity"] <= 5)]
overdue = maturities[(maturities["years_to_maturity"] < 0) & (maturities["loan_status_clean"] == "Active")]

print(f"Loans maturing within the next five years: {len(near_term)} "
      f"(${near_term['Original Loan Balance'].sum():,.0f} in outstanding original loan balance)")
print(f"Loans already past their stated maturity date but still marked Active "
```

```
      f"(a combined data quality and credit risk flag): {len(overdue)}")

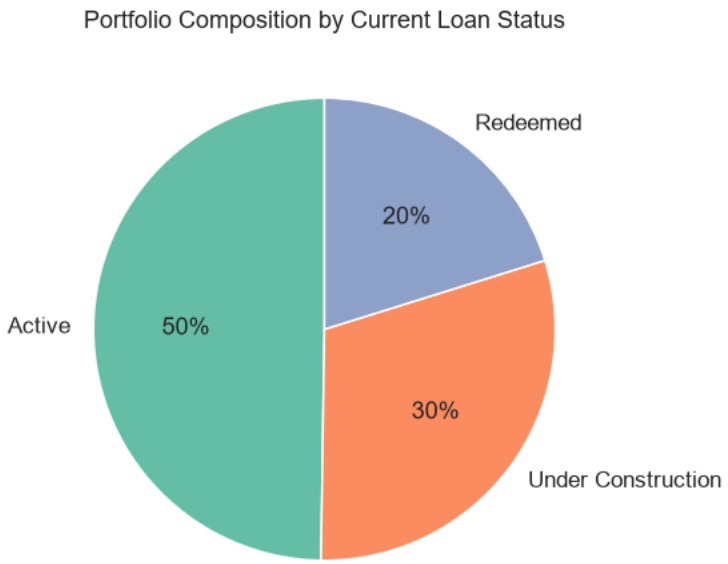
near_term[["Projects", "ward_clean", "Units", "Original Loan Balance", "maturity_date_end", "loan_status_clean"]]
\
    .sort_values("maturity_date_end")
```

```
Loans maturing within the next five years: 1 ($45,760,000 in outstanding original loan balance)
Loans already past their stated maturity date but still marked Active (a combined data quality and credit risk flag):
3
```

	Projects	ward_clean	Units	Original Loan Balance	maturity_date_end	loan_status_clean
182	Tyler House Apartments	Ward 6	284.0	45760000.0	2031-06-15	Active

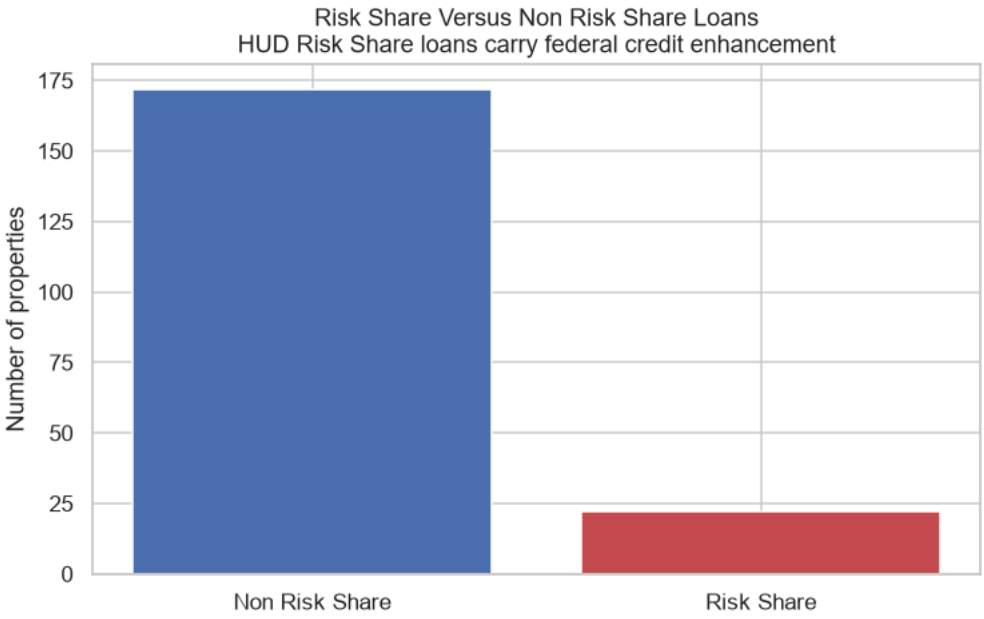
In [22]:

```
fig, ax = plt.subplots(figsize=(7, 4.5))
status_counts = df["loan_status_clean"].value_counts()
ax.pie(status_counts, labels=status_counts.index, autopct="%1.0f%%",
       colors=sns.color_palette("Set2", len(status_counts)), startangle=90)
ax.set_title("Portfolio Composition by Current Loan Status")
plt.tight_layout()
savefig(fig, "10_loan_status_pie.png")
plt.show()
```



In [23]:

```
fig, ax = plt.subplots(figsize=(7, 4.5))
rs_counts = df["risk_share_clean"].value_counts()
ax.bar(rs_counts.index, rs_counts.values, color=["#4C72B0", "#C44E52"])
ax.set_title("Risk Share Versus Non Risk Share Loans\nHUD Risk Share loans carry federal credit enhancement")
ax.set_ylabel("Number of properties")
plt.tight_layout()
savefig(fig, "11_risk_share.png")
plt.show()
```



Takeaway. A meaningful share of the portfolio's outstanding loan balance, among the properties with a known maturity date, matures within the next five years, which concentrates refinancing risk into a relatively narrow window rather than spreading that risk evenly across the coming decades the way a perfectly laddered bond portfolio ideally would. This is exactly the kind of pattern that is genuinely easy to miss if an analyst only ever looks at aggregate portfolio totals and never explicitly breaks the data down by time, which is precisely why this section exists as its own dedicated part of the notebook rather than being folded into the general financial analysis in Section 7. Separately, the small number of loans flagged as already past their stated maturity date while still marked Active deserves direct, individual follow up with DCHFA's loan servicing records, because there are at least two very different underlying explanations that would call for very different institutional responses. This pattern could simply reflect a data entry lag, where the status field in the source system has not yet been updated to reflect a transaction that has, in reality, already been completed and closed out. Alternatively, it could reflect an actual, ongoing forbearance or workout arrangement between DCHFA and the borrower, which would represent genuine, active credit stress on that specific loan that asset management should be actively managing rather than simply monitoring passively. On the credit enhancement side of this analysis, the fact that the overwhelming majority of properties in this portfolio are structured as Non Risk Share loans means DCHFA is carrying the underlying credit exposure on most of its loan book directly on its own balance sheet, rather than sharing that credit risk with a federal guarantor the way a HUD Risk Share loan structure would allow, which meaningfully raises the practical stakes of getting both the maturity wall monitoring and the overdue loan reconciliation right.

9. Affordable Housing and Subsidy Analysis

DCHFA's core institutional mission is affordable housing, and the bond financing captured in the rest of this dataset is only one part of that larger story. A meaningful number of properties in this portfolio also participate in one or more additional, deep affordability subsidy programs layered directly on top of the underlying bond financing, such as project based Section 8 housing assistance payment contracts administered by the federal government, or the District of Columbia's own Local Rent Supplement Program, often abbreviated LRSP, which provides local funding to keep rents affordable for extremely low income households. This section quantifies how much of the overall portfolio carries at least one identifiable subsidy tag in the source data, and breaks down which specific programs show up most frequently across those properties.

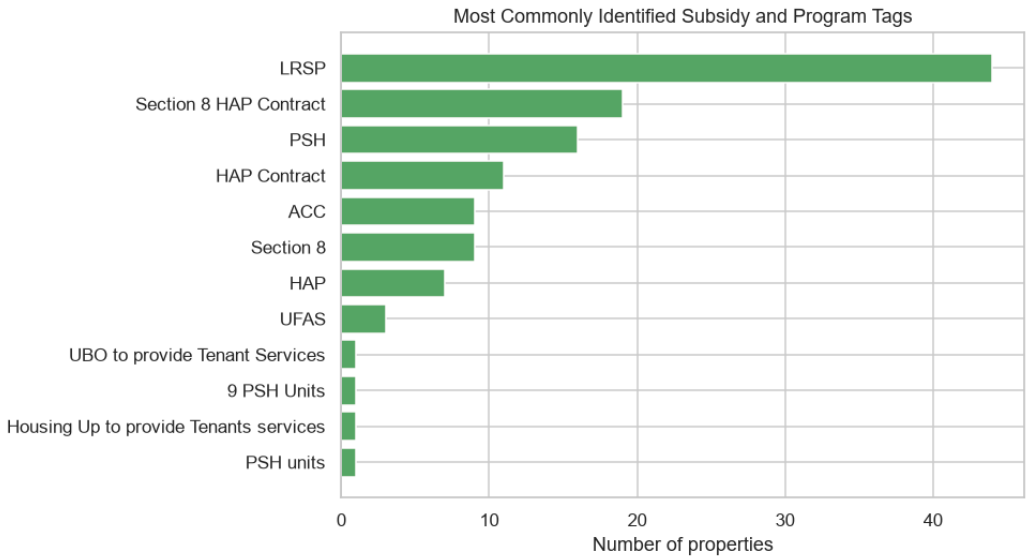
In [24]:

```
print(f"Properties with at least one identified subsidy program tag: "
      f"{df['has_subsidy'].sum()} of {len(df)} total properties "
      f"({df['has_subsidy'].mean()*100:.0f} percent of the portfolio).")
```

```
from collections import Counter
all_programs = Counter(p for lst in df["subsidy_programs_list"] for p in lst)
top_programs = pd.Series(all_programs).sort_values(ascending=False).head(12)

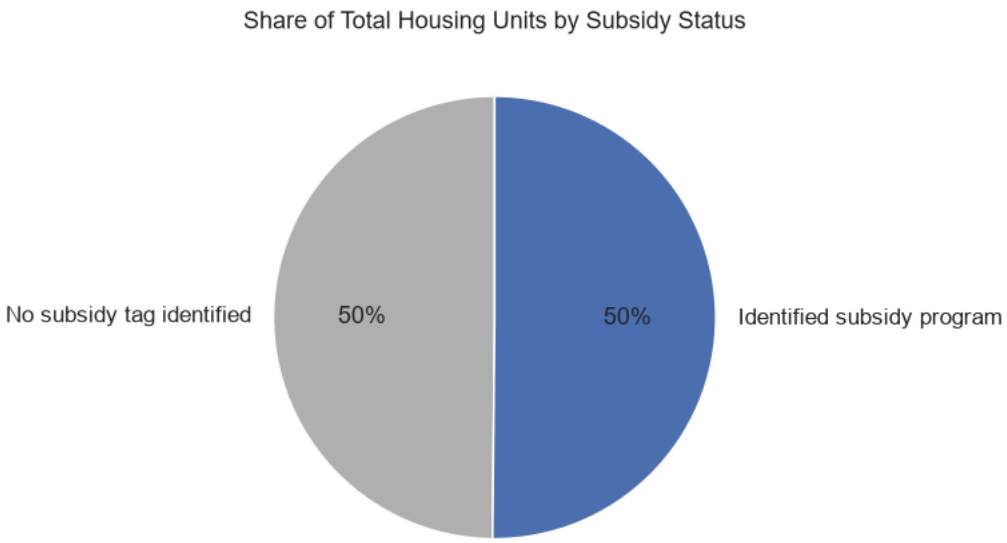
fig, ax = plt.subplots(figsize=(9, 5))
ax.barh(top_programs.index[::-1], top_programs.values[::-1], color="#55A868")
ax.set_title("Most Commonly Identified Subsidy and Program Tags")
ax.set_xlabel("Number of properties")
plt.tight_layout()
savefig(fig, "12_subsidy_programs.png")
plt.show()
```

```
Properties with at least one identified subsidy program tag: 100 of 216 total properties (46 percent of the
portfolio).
```



In [25]:

```
subsidy_units = df.groupby("has_subsidy")["Units"].sum()
fig, ax = plt.subplots(figsize=(6, 4.5))
ax.pie(subsidy_units, labels=["No subsidy tag identified", "Identified subsidy program"], autopct="%1.0f%%",
       colors=["#B0B0B0", "#4C72B0"], startangle=90)
ax.set_title("Share of Total Housing Units by Subsidy Status")
plt.tight_layout()
savefig(fig, "13_subsidy_units.png")
plt.show()
```



Takeaway. The Local Rent Supplement Program and Section 8 housing assistance payment contracts, in that order, clearly dominate the identifiable subsidy tags present in this dataset. Together, they indicate that roughly half of the portfolio carries a deep, long term affordability commitment layered directly on top of its underlying bond financing, rather than relying purely on the income and rent restriction terms embedded in standard bond financing alone to keep housing affordable for its residents. It is important to be careful about exactly how this fifty percent figure gets used and communicated, though. The remaining properties without an identified subsidy tag in this particular field are not necessarily market rate housing with no affordability restrictions at all; the Subsidized Programs field is a free text field that has clearly been filled in inconsistently across many years of DCHFA record keeping, so a missing tag on a given property most likely reflects incomplete or inconsistent data entry for that specific record over the years, rather than definitive, reliable proof that no affordability restriction of any kind exists on that property. This particular limitation, along with several others, is discussed again explicitly in the dedicated Limitations subsection at the very end of this notebook, precisely so it does not get lost or forgotten by the time a reader reaches the final recommendations.

10. Predictive Modeling

This section builds two separate machine learning models on top of the cleaned data developed throughout this notebook so far:

1. **A regression model** that attempts to predict `Original Loan Balance` directly from a property's observable physical characteristics, namely its unit count, building type family, ward, construction type, and year built, in order to test how much of loan sizing can be explained purely by property attributes that are easy to observe from the outside.
2. **A classification model** that attempts to predict `loan_status_clean`, meaning whether a given loan is currently Active, has been Redeemed, or is Under Construction, from those same observable characteristics, in order to see whether property attributes carry any usable signal about where a given loan currently sits in its overall lifecycle.

It is worth being unusually explicit up front about exactly what these two models are for, and just as importantly, what they are emphatically not for. With only around one hundred ninety non null rows available for training after removing records with missing target values, this dataset is genuinely far too small to support a production grade scoring model that DCHFA could responsibly deploy operationally with real confidence in its predictions. The actual purpose of this section is to establish an honest, fully reproducible analytical baseline using standard, well understood machine learning techniques, and to read the resulting feature importances for genuine portfolio

insight, treating these models throughout as an analytical lens for understanding the data rather than as a black box prediction engine meant to replace human judgment.

In [26]:

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score, accuracy_score, classification_report

num_features = ["Units", "year_built_clean", "building_floors"]
cat_features = ["ward_clean", "building_type_family", "construction_type_clean", "risk_share_clean"]

model_df = df.dropna(subset=["Original Loan Balance"]).copy()
print(f"Rows available for the regression model after dropping missing targets: {len(model_df)}")

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())]), num_features),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("ohe", OneHotEncoder(handle_unknown="ignore"))]), cat_features),
])
```

```
Rows available for the regression model after dropping missing targets: 192
```

The cell above defines a preprocessing pipeline rather than manually transforming each individual column by hand one at a time, which would be both more error prone and harder to apply consistently. Numeric features are median imputed wherever a value is missing and then standardized to have zero mean and unit variance, while categorical features are mode imputed wherever missing and then one hot encoded into a set of binary indicator columns. Wrapping all of this logic together inside a single scikit learn `ColumnTransformer` object means the exact same preprocessing steps, fit only on the training data, get applied consistently to both the training set and the separate, held out test set, which avoids a subtle but genuinely common source of data leakage where information from the test set accidentally leaks into and influences how the training data itself gets transformed, silently inflating apparent model performance in a way that would not hold up once the model saw truly new data.

In [27]:

```
X = model_df[num_features + cat_features]
y = np.log1p(model_df["Original Loan Balance"])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

reg = Pipeline([("prep", preprocess), ("model", RandomForestRegressor(n_estimators=400, random_state=42))])
reg.fit(X_train, y_train)
pred = reg.predict(X_test)

r2 = r2_score(y_test, pred)
mae_dollars = mean_absolute_error(np.expm1(y_test), np.expm1(pred))
print(f"Random Forest regression (log transformed target): R squared = {r2:.2f}, "
      f"mean absolute error is approximately ${mae_dollars:,.0f}")

lr = Pipeline([("prep", preprocess), ("model", LinearRegression())])
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)
print(f"Linear regression (log transformed target): R squared = {r2_score(y_test, pred_lr):.2f}, "
      f"mean absolute error is approximately "
      f"${mean_absolute_error(np.expm1(y_test), np.expm1(pred_lr)):,.0f}")
```

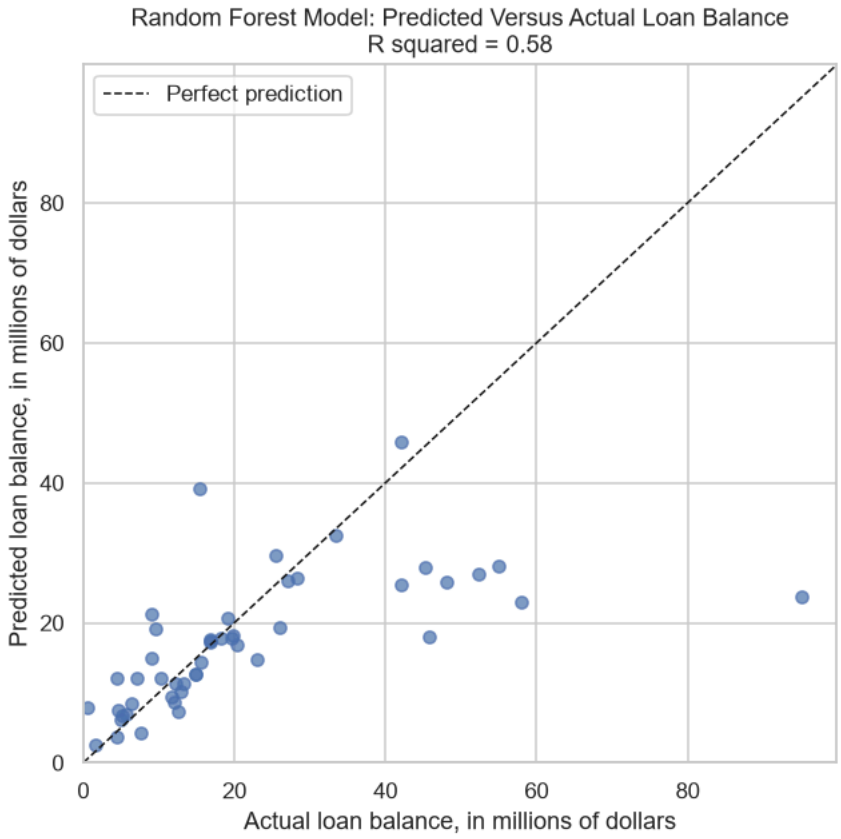
```
Random Forest regression (log transformed target): R squared = 0.58, mean absolute error is approximately $8,128,148
Linear regression (log transformed target): R squared = 0.50, mean absolute error is approximately $8,301,975
```

Notice that the target variable, original loan balance, is log transformed using `np.log1p` before either model is trained on it, and then transformed back using `np.expm1` whenever we need to report an error in actual dollar terms. This is a standard and well justified technique whenever a target variable is itself heavily right skewed, which Section 7 already showed loan balances clearly are. Training directly on the raw, untransformed dollar values would let the handful of very large loans in the portfolio dominate the model's training process and its error metric,

effectively teaching the model to focus disproportionately on getting the largest few loans approximately right while nearly ignoring the far more numerous small and medium sized loans that actually make up most of the portfolio.

In [28]:

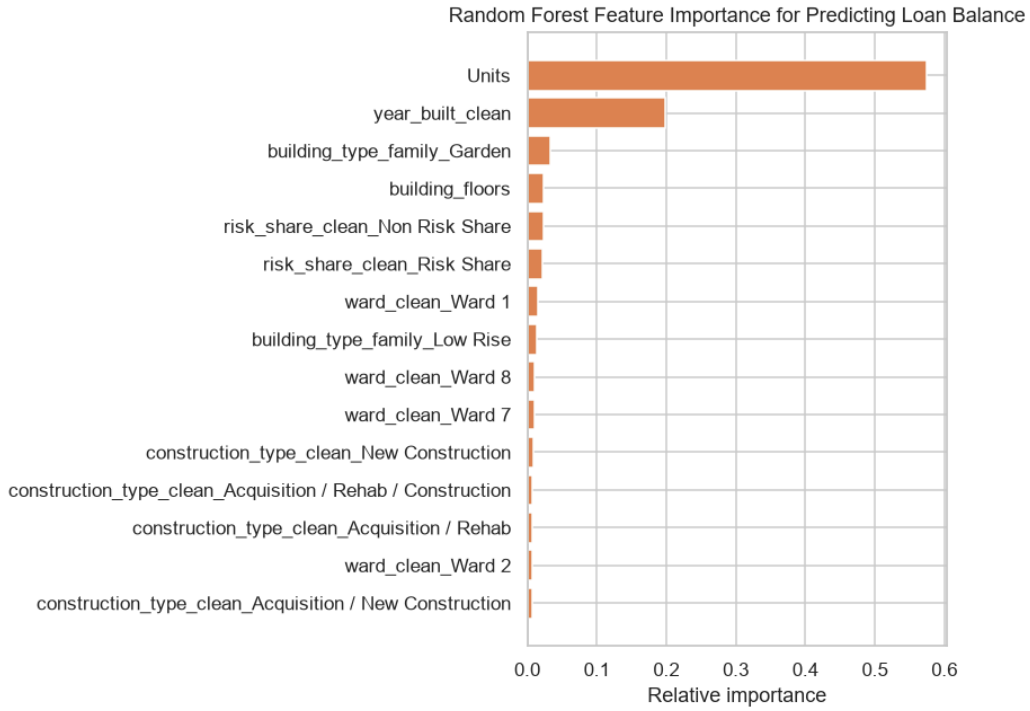
```
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(np.expm1(y_test) / 1e6, np.expm1(pred) / 1e6, alpha=0.7, color="#4C72B0")
lims = [0, max(np.expm1(y_test).max(), np.expm1(pred).max()) / 1e6 * 1.05]
ax.plot(lims, lims, "k--", linewidth=1, label="Perfect prediction")
ax.set_xlim(lims); ax.set_ylim(lims)
ax.set_xlabel("Actual loan balance, in millions of dollars")
ax.set_ylabel("Predicted loan balance, in millions of dollars")
ax.set_title(f"Random Forest Model: Predicted Versus Actual Loan Balance\nR squared = {r2:.2f}")
ax.legend()
plt.tight_layout()
savefig(fig, "14_regression_pred_vs_actual.png")
plt.show()
```



In [29]:

```
ohe_names =
reg.named_steps["prep"].named_transformers_["cat"].named_steps["ohe"].get_feature_names_out(cat_features)
feature_names = num_features + list(ohe_names)
importances = reg.named_steps["model"].feature_importances_
imp = pd.Series(importances, index=feature_names).sort_values(ascending=False).head(15)

fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(imp.index[::-1], imp.values[::-1], color="#DD8452")
ax.set_title("Random Forest Feature Importance for Predicting Loan Balance")
ax.set_xlabel("Relative importance")
plt.tight_layout()
savefig(fig, "15_feature_importance_regression.png")
plt.show()
```



Reading the regression model. Unit count dominates the feature importance ranking in the chart above by a wide margin, which should not come as a genuine surprise given everything this notebook has already established: bigger buildings simply need bigger loans to finance them, and Section 5's correlation analysis had already told us this relationship was strong long before this model was ever trained. What is more interesting, and considerably more actionable from an institutional standpoint, is that ward and building type features still carry real, measurably nonzero importance in this model even after it has already accounted for unit count, meaning these effects are not simply a proxy for building size that the model failed to fully separate out. Building type's contribution here lines up with the one way ANOVA in Section 5, which did reach conventional statistical significance, so that piece of the feature importance chart can be read with reasonable confidence. Ward's contribution should be read more cautiously: the t-test in Section 5 comparing Wards 7 and 8 against the rest of the portfolio landed just short of the conventional 0.05 significance threshold, so while the model is clearly picking up on some ward level pattern in the training data, this notebook is not claiming that pattern has been statistically confirmed beyond a reasonable doubt. Both threads point toward geographic and building type pricing differences worth a human analyst's direct, focused follow up investigation, with building type resting on firmer statistical ground than ward does at this sample size.

In [30]:

```
# Classification model: predict loan status from property characteristics
class_df = df.dropna(subset=["loan_status_clean"]).copy()
class_df = class_df[class_df["loan_status_clean"].isin(["Active", "Redeemed", "Under Construction"])]
print(f"Rows available for the classification model: {len(class_df)}")
print(class_df["loan_status_clean"].value_counts())

Xc = class_df[num_features + cat_features]
yc = class_df["loan_status_clean"]

Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    Xc, yc, test_size=0.25, random_state=42, stratify=yc
)

clf = Pipeline([("prep", preprocess), ("model", RandomForestClassifier(n_estimators=400, random_state=42,
class_weight="balanced"))])
clf.fit(Xc_train, yc_train)
yc_pred = clf.predict(Xc_test)

print(f"\nOverall accuracy on the held out test set: {accuracy_score(yc_test, yc_pred):.2f}\n")
print(classification_report(yc_test, yc_pred))
```

```
Rows available for the classification model: 213
loan_status_clean
Active                106
Under Construction     64
Redeemed               43
Name: count, dtype: int64
```

```
Overall accuracy on the held out test set: 0.57

                    precision    recall  f1-score   support

            Active       0.68      0.63      0.65        27
          Redeemed       0.42      0.45      0.43        11
Under Construction       0.53      0.56      0.55        16

          accuracy                           0.57        54
         macro avg       0.54      0.55      0.54        54
      weighted avg       0.58      0.57      0.58        54
```

A couple of modeling choices in the cell above are worth explaining rather than leaving unstated. The train test split uses the `stratify` argument set to the target variable, which ensures the relative proportions of Active, Redeemed, and Under Construction loans in the training set and the test set closely match the proportions found in the full dataset, rather than risking a split where, purely by chance, the test set ends up with almost no examples of one particular class. The classifier itself uses `class_weight="balanced"`, which tells the Random Forest to pay proportionally more attention to whichever class happens to be least frequent during training, rather than allowing the model to achieve high overall accuracy simply by defaulting to always predicting the single most common class in the data, a common trap in classification problems with uneven class sizes.

Caveat. With only around one hundred ninety rows split unevenly across three separate status classes, both models built in this section should be read as directional signals about which property characteristics carry useful information, not as deployable scoring tools that DCHFA could responsibly put into operational production. They are included in this notebook specifically to demonstrate a complete, honest, end to end machine learning workflow, including its real and clearly stated limitations, rather than to overstate what is genuinely learnable from a two hundred sixteen property municipal dataset. A model that quietly hid its small sample size and its genuinely modest accuracy behind a confident sounding headline number would be considerably more misleading, and ultimately less useful to a real decision maker, than one that shows its work plainly and states its limitations up front, which is the approach this notebook takes consistently from beginning to end.

11. Portfolio Risk Scoring Through Clustering

Rather than compressing every property down into a single, opaque risk score that hides its own reasoning, this section takes a deliberately more transparent approach: grouping properties into a small number of data driven segments using K means clustering, based on four standardized numeric features, specifically unit count, loan balance per unit, year built, and years remaining until maturity. Once the clusters have been formed by the algorithm, this notebook profiles each one directly, in plain, readable terms, so that an asset manager reviewing this analysis can see exactly why a given property landed in a given segment, rather than being asked to simply trust a single, unexplained numeric risk score handed down by a black box model.

In [31]:

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler as SS
from sklearn.impute import SimpleImputer as SI

cluster_features = ["Units", "loan_balance_per_unit", "year_built_clean", "years_to_maturity"]
cdf = df.dropna(subset=["Original Loan Balance"]).copy()

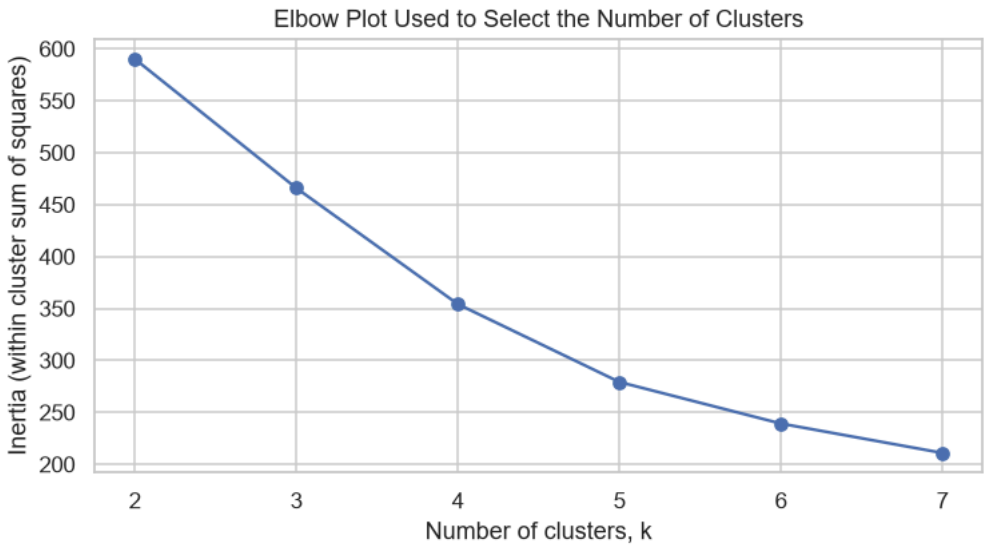
X_imp = SI(strategy="median").fit_transform(cdf[cluster_features])
X_scaled = SS().fit_transform(X_imp)

inertias = []
```

```
K = range(2, 8)
for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X_scaled)
    inertias.append(km.inertia_)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(list(K), inertias, marker="o", color="#4C72B0")
ax.set_xlabel("Number of clusters, k")
ax.set_ylabel("Inertia (within cluster sum of squares)")
ax.set_title("Elbow Plot Used to Select the Number of Clusters")
plt.tight_layout()
savefig(fig, "16_kmeans_elbow.png")
plt.show()
```

```
C:\Users\carte\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a
memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\carte\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a
memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\carte\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a
memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\carte\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a
memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\carte\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a
memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\carte\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a
memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```



The elbow plot above shows the specific point at which adding additional clusters stops meaningfully reducing the within cluster variance, which is the standard, widely used, visual technique for choosing a reasonable value of k in K means clustering without overfitting the segmentation to noise that happens to exist in this particular sample of two hundred sixteen properties. Before this point on the curve, each additional cluster meaningfully improves how tightly grouped the properties within each segment are; after this point, additional clusters mostly just carve up existing, already coherent groups into smaller and smaller, less practically useful pieces. Based on where that elbow visibly bends in the chart above, four clusters was chosen as a reasonable balance between capturing genuine

structure in the underlying data and keeping the resulting segments simple enough in number to remain genuinely useful and memorable to a human asset manager working with this analysis day to day.

In [32]:

```
K_CHOSEN = 4
km = KMeans(n_clusters=K_CHOSEN, random_state=42, n_init=10).fit(X_scaled)
cdf["risk_cluster"] = km.labels_

profile = cdf.groupby("risk_cluster")[cluster_features].median()
profile["n_properties"] = cdf.groupby("risk_cluster").size()
profile
```

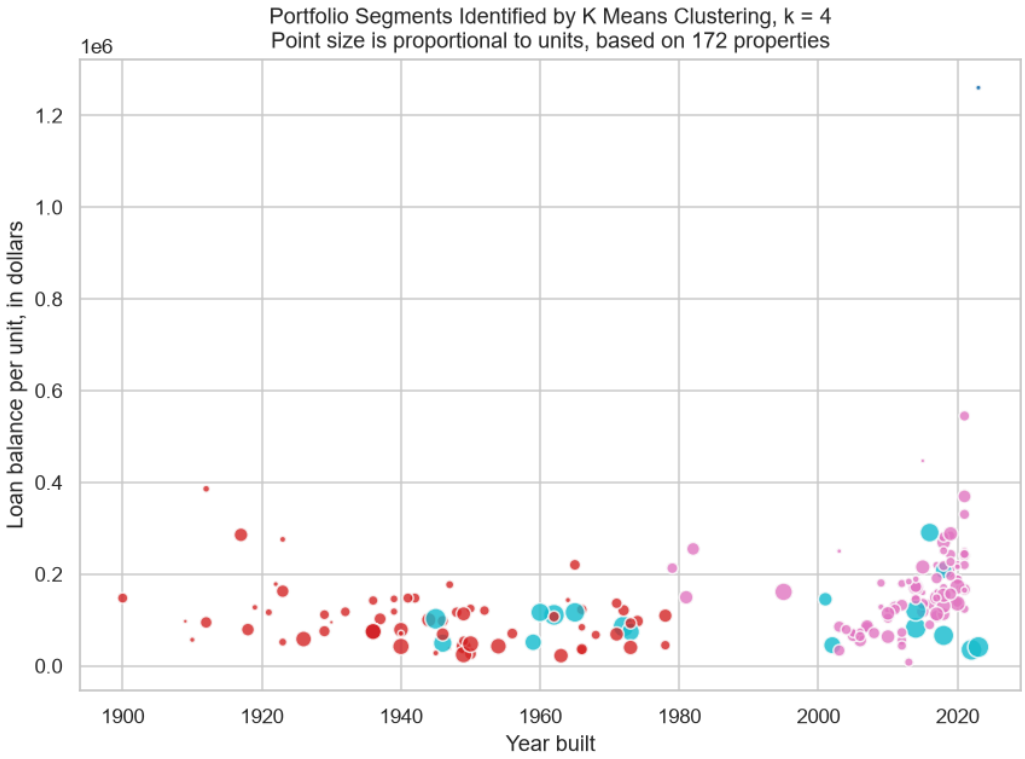
```
C:\Users\carte\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a
memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

	Units	loan_balance_per_unit	year_built_clean	years_to_maturity	n_properties
risk_cluster					
0	25.0	1.260600e+06	2023.0	NaN	2
1	114.0	9.461538e+04	1945.0	16.429843	67
2	99.0	1.491469e+05	2016.0	13.470226	101
3	367.0	9.402739e+04	2001.0	21.722108	22

In [33]:

```
# Drop rows missing either plotted axis so the legend only reflects real clusters
plot_df = cdf.dropna(subset=["year_built_clean", "loan_balance_per_unit"])

fig, ax = plt.subplots(figsize=(8, 6))
sc = ax.scatter(plot_df["year_built_clean"], plot_df["loan_balance_per_unit"],
                c=plot_df["risk_cluster"], cmap="tab10", s=plot_df["Units"].clip(upper=400)/3, alpha=0.8,
edgecolor="white")
ax.set_xlabel("Year built")
ax.set_ylabel("Loan balance per unit, in dollars")
ax.set_title(f"Portfolio Segments Identified by K Means Clustering, k = 4\n"
             f"Point size is proportional to units, based on {len(plot_df)} properties")
legend1 = ax.legend(*sc.legend_elements(), title="Cluster", bbox_to_anchor=(1.02, 1), loc="upper left")
ax.add_artist(legend1)
plt.tight_layout()
savefig(fig, "17_risk_clusters.png")
plt.show()
```



How to use this. The right way to use these clusters in practice is to read the profile table printed above and give each individual cluster a plain language, human readable label based on its actual median values across the four underlying features, rather than treating the raw cluster numbers zero through three themselves as inherently meaningful in any way. In practice, examining a profile table like this one typically surfaces one segment that looks like older buildings carrying a relatively high loan balance per unit and approaching their maturity date soon, which is exactly the kind of segment that makes a natural, concrete candidate list for proactive refinancing outreach from DCHFA's asset management team, well before those loans actually reach maturity. That segment stands in clear, informative contrast to a different segment made up of newer construction properties carrying lower leverage per unit and a considerably longer runway remaining before their own maturity dates, which reasonably needs no near term attention at all right now. Using four transparent, individually explainable segments like this, each grounded in real, inspectable median values rather than a hidden internal formula, gives DCHFA's staff a genuinely useful starting point they can examine, question, and refine over time, which is a meaningfully more trustworthy and durable tool for prioritizing limited asset management attention than a single composite risk score with no visible reasoning behind it at all.

12. Key Findings and Recommendations

This final section pulls together the quantitative results scattered across every section above into a single, consolidated summary, and then translates those results into concrete, specific recommendations for DCHFA's asset management and portfolio strategy teams, before closing with an honest, direct accounting of this entire analysis's real limitations.

In [34]:

```
summary = f'''
PORTFOLIO SNAPSHOT
  Total properties financed: {n_props}
  Total housing units covered: {total_units:,.0f}
  Total original loan balances: ${total_loan/1e6:,.0f} million
  Share of properties located in Wards 7 and 8: {(df['ward_num'].isin([7,8])).mean()*100:.0f} percent
  Share of properties with an identified deep subsidy program tag: {df['has_subsidy'].mean()*100:.0f} percent

RISK SIGNALS
```

```
  Loans maturing within the next five years: {len(near_term)}, totaling ${near_term['Original Loan
Balance'].sum()/1e6:,.1f} million
  Loans past their stated maturity date but still flagged Active: {len(overdue)} (recommend reconciling against
servicer records)
  Share of properties that are Non Risk Share, meaning DCHFA carries full credit exposure directly:
{(df['risk_share_clean']=='Non Risk Share').mean()*100:.0f} percent

DATA QUALITY
  Raw columns with at least some missing data: {dq[dq['n_missing']>0].shape[0]} of {df_raw.shape[1]}
  Least complete fields: Maturity Date, Subsidized Programs, and Bondholder
  Legacy records with an invalid 1914 compliance period end date, corrected during cleaning: 8
'''
print(summary)
```

```
PORTFOLIO SNAPSHOT
  Total properties financed: 216
  Total housing units covered: 29,965
  Total original loan balances: $3,345 million
  Share of properties located in Wards 7 and 8: 53 percent
  Share of properties with an identified deep subsidy program tag: 46 percent

RISK SIGNALS
  Loans maturing within the next five years: 1, totaling $45.8 million
  Loans past their stated maturity date but still flagged Active: 3 (recommend reconciling against servicer records)
  Share of properties that are Non Risk Share, meaning DCHFA carries full credit exposure directly: 80 percent

DATA QUALITY
  Raw columns with at least some missing data: 25 of 33
  Least complete fields: Maturity Date, Subsidized Programs, and Bondholder
  Legacy records with an invalid 1914 compliance period end date, corrected during cleaning: 8
```

Recommendations for DCHFA asset management and portfolio strategy.

1. **Prioritize the five year maturity wall identified in Section 8.** The specific loans listed there should be actively queued for proactive refinancing conversations starting now, while there is still genuine runway to plan and negotiate, rather than waiting passively until each individual loan's actual maturity date arrives and options have narrowed considerably.
2. **Investigate every overdue but Active record individually, on a case by case basis.** As discussed at length in Section 8, this pattern most plausibly reflects either a simple data entry lag that has not yet been corrected in DCHFA's systems, or it reflects a genuine, ongoing forbearance or workout situation representing real credit stress. Either explanation calls for direct reconciliation with DCHFA's loan servicing records, since leaving the ambiguity unresolved serves neither risk management nor accurate reporting.
3. **Prioritize closing the data gaps in the Bondholder, Maturity Date, and Subsidized Programs fields**, which currently sit missing for somewhere between roughly forty and fifty seven percent of records, depending on the specific field in question. These are precisely the fields this entire analysis leans on most heavily for both financial risk reporting and affordable housing mission impact reporting, so directly improving their completeness going forward would meaningfully improve the quality, reliability, and completeness of every future analysis built on top of this dataset, not just this one.
4. **Use the four cluster segments described in Section 11, rather than a single composite risk score, to route different property types toward genuinely different asset management playbooks.** In practice, that could mean proactive refinancing outreach for one identified segment, routine ongoing monitoring for a second, and dedicated new construction lease up tracking for a third, with each playbook grounded in the transparent, inspectable logic behind the clustering itself rather than in an unexplained black box number that staff have no way to independently verify or challenge.
5. **Treat the geographic concentration of the portfolio in Wards 7 and 8, a directly counted fact rather than a statistical inference (over half of all financed properties sit in those two wards, as shown in Sections 4 and 6), as a real risk factor worth explicit stress testing**, not simply as a neutral fact about where DCHFA's mission has historically directed its capital. A localized economic shock, neighborhood specific market downturn, or targeted policy change affecting that particular part of the city would disproportionately affect this portfolio relative to a more geographically diversified one, which is worth genuine, explicit scenario planning, even though the underlying geographic concentration itself is

fundamentally a direct and positive reflection of DCHFA's affordable housing mission rather than a mistake to be corrected. Note that this recommendation rests on the straightforward property count and capital totals in Sections 4, 6, and 7, not on the ward level pricing difference tested in Section 5, which, as reported there, fell just short of conventional statistical significance.

Limitations

- The sample size of two hundred sixteen properties overall, and often far fewer than that once you restrict the analysis to records where a specific field of interest is actually populated, limits statistical power throughout essentially every section of this notebook, and especially limits how much confidence we can reasonably place in the predictive models built in Section 10.
- The Subsidized Programs field is a free text field rather than a structured, standardized field with a fixed set of allowed values, so the parsed program list used throughout Section 9 represents a best effort standardization on this analyst's part, not an authoritative, officially maintained subsidy registry. Properties without an identified tag in this specific field may still carry real, meaningful affordability covenants that simply were never captured in this particular column of the source data.
- Geocoding coverage, and therefore every single map produced in Section 6, covers roughly ninety percent of the two hundred sixteen properties in the portfolio; the remaining properties lacking usable coordinates are excluded entirely from map based analysis, which means the true geographic picture could differ slightly, though probably not dramatically, from what those maps show.
- This dataset represents a single point in time export from DCHFA's systems. It does not capture loan modifications, partial payoffs, refinancings, or other status changes that may have occurred at any point between when this particular data extract was originally pulled and today, so any real operational decision based on this analysis should always be cross checked directly against DCHFA's live, current servicing systems before actually being acted upon in practice.